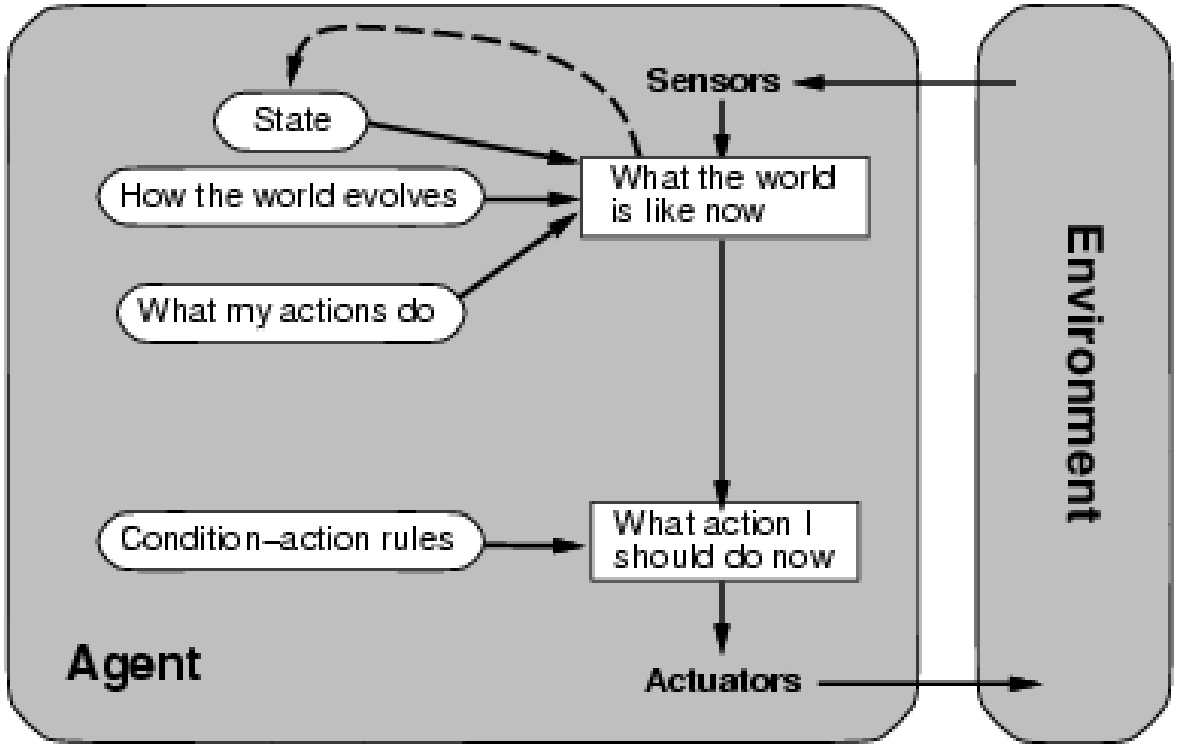
Artificial Intelligence (CS1360)
Searching Methodology

Introduction



How can we represent the environment that the agent inhabits?

Introduction

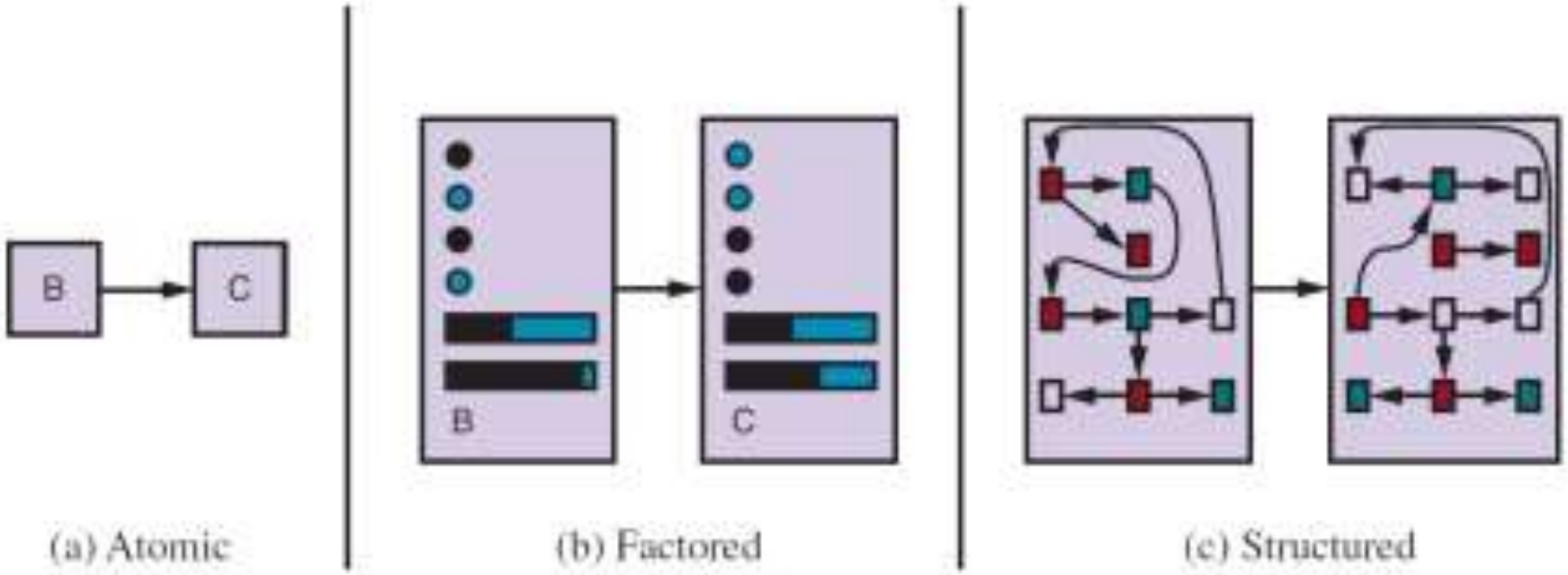
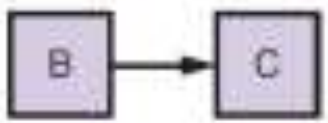


Figure 2.16 Three ways to represent states and the transitions between them. (a) Atomic representation: a state (such as B or C) is a black box with no internal structure; (b) Factored representation: a state consists of a vector of attribute values; values can be Boolean, real-valued, or one of a fixed set of symbols. (c) Structured representation: a state includes objects, each of which may have attributes of its own as well as relationships to other objects.

Problem-



(a) Atomic

The problem-solving agent

- is one kind of goal-based agents
- decides what to do by finding a sequence of actions that leads to a desirable states (goals).
- It uses atomic representation!

Simple Problem-solving agents : environment type

- Environment type : static (the problem is formulated and solved without paying any attention to the change of the environment).
- The environment is observable : the initial state is assumed known (it is easiest if the environment is observable).
- The environment can be viewed as discrete since the actions at each state can be enumerated.
- The environment is deterministic (each action leads to a unique state).

Problem-Solving Process

The agent can follow this four-phase problem-solving process:

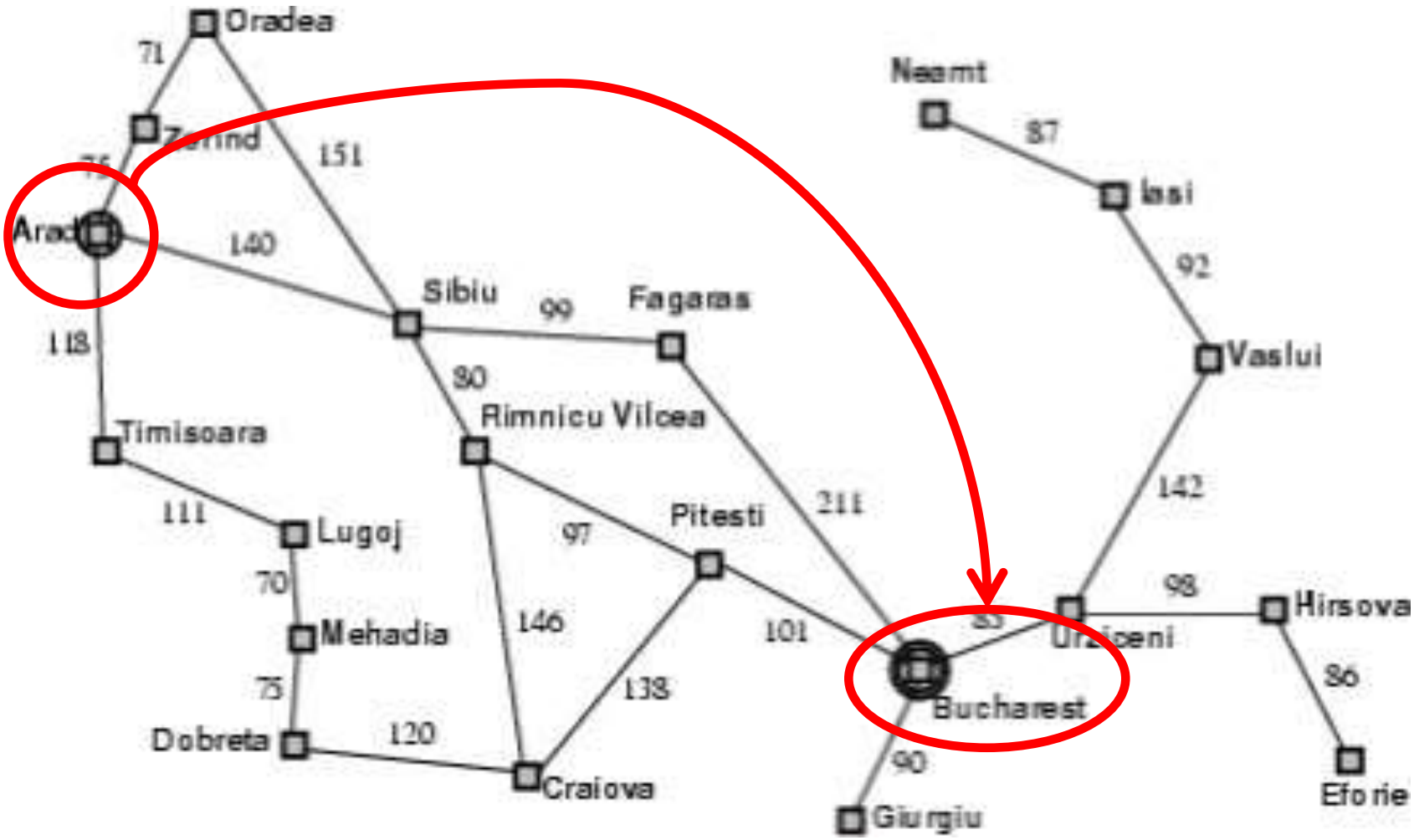
1. Goal formulation.
2. Problem formulation: a description of the states and actions—an abstract model of the relevant part of the world.
3. Search: simulating sequences of actions in the model, until finding a solution.
4. Execution the actions in the solution.

Search Approach

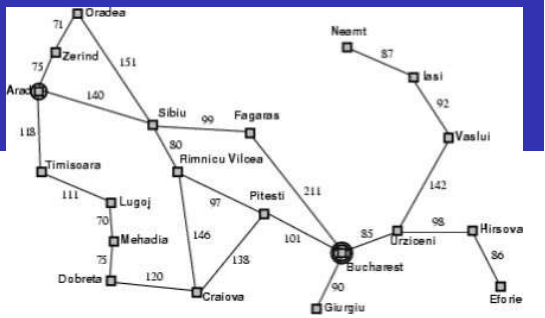
- The search algorithm takes a problem as an input and returns a solution in form of an action sequence.

- Why searching methodology in AI?
  - Solving difficult problems with a computer NP-Complete, NP-hard problems.
  - Develop efficient generic algorithms for solving these problems.

Example: Romania



A well-defined problem and solutions



A well-defined problem is defined as follows:

State space: a set of possible states that the environment can be in.

Initial state : a given current state the agent starts in.

Goal states: one or more goal states defined by a goal test.

Actions: the set of actions that can be executed in a particular state.

Transition model: describing what each action does. RESULT(s, a) returns the state that results from doing action a in state s.

A well-defined problem and solutions

An action cost function: a numeric cost of each action, *ACTION-COST(s,a,s')*, that reflects the agent's performance measure.

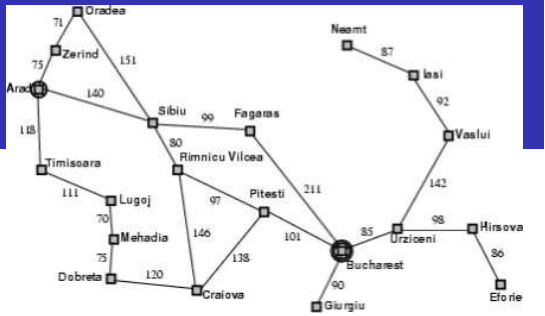
A **path** in the state space is a sequence of states connected by a sequence of actions.

Path cost is the sum of the individual action costs.

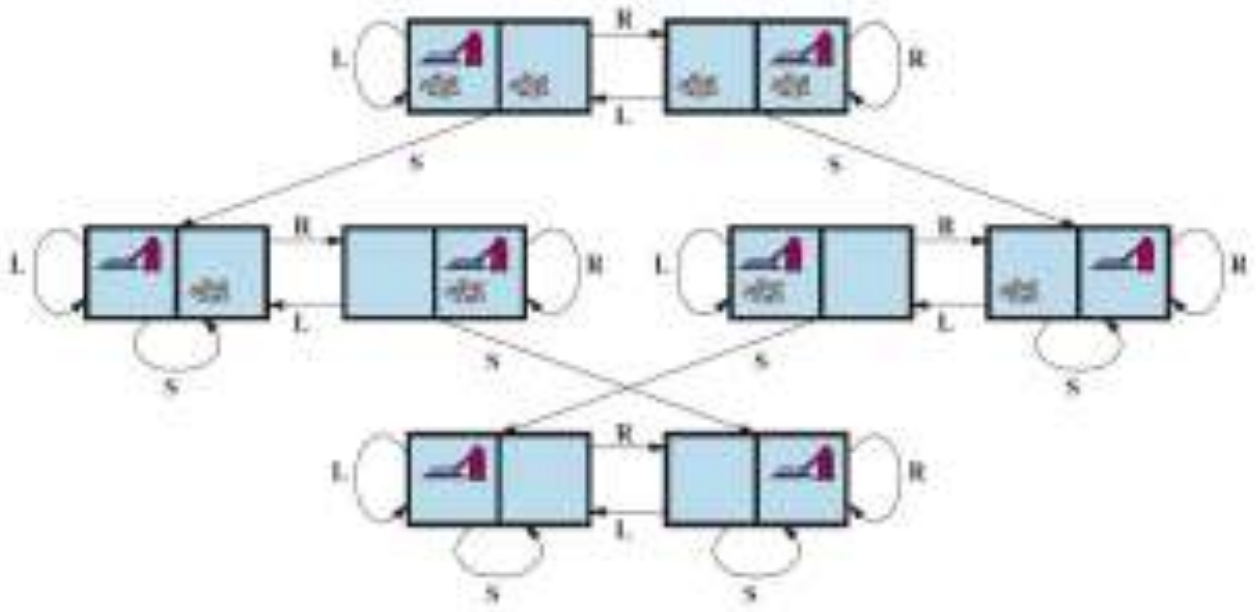
A **solution** to the problem is a path from the initial state to a goal state.

Solution quality is measured by the path cost, and the optimal solution is the lowest path cost among all solutions.

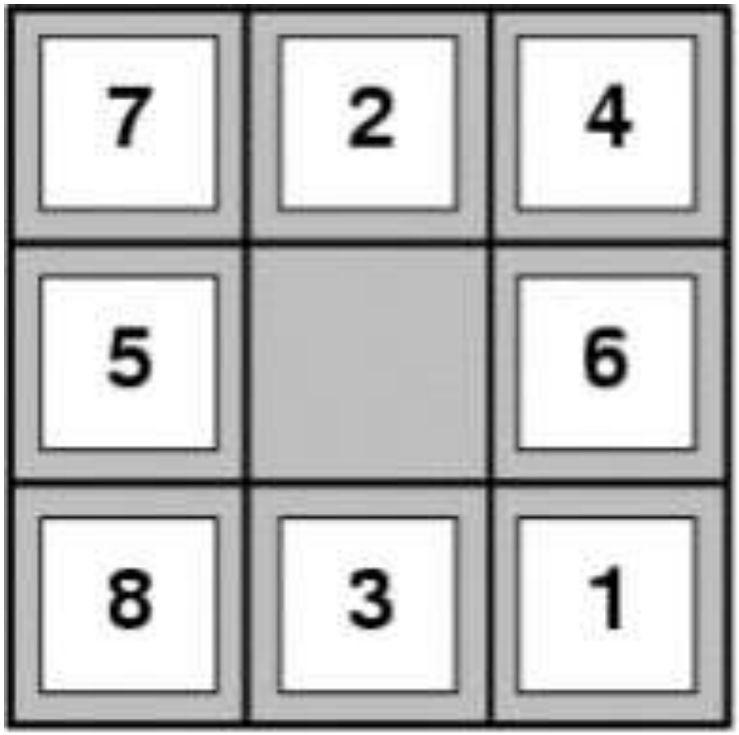
A well-defined problem and solutions



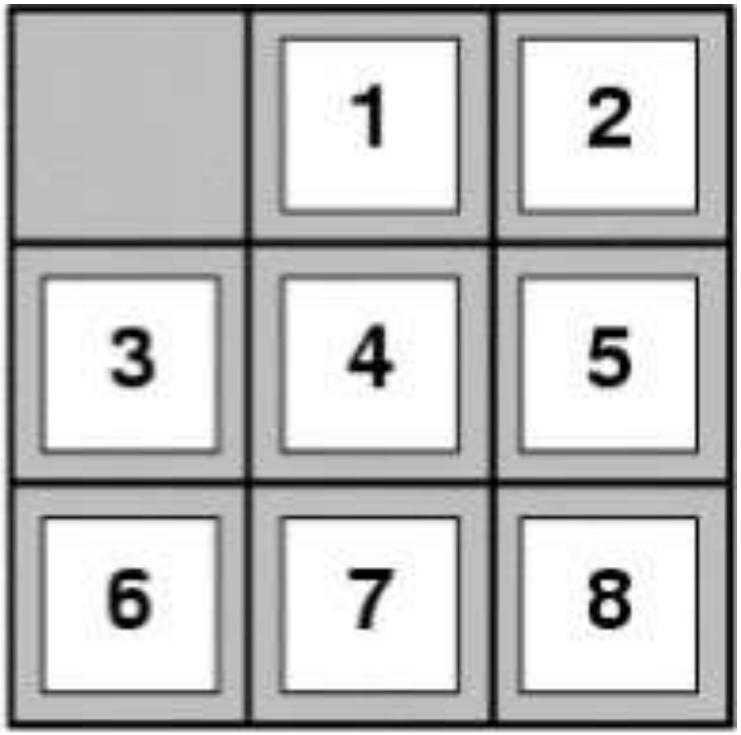
The state space of a discrete problems can be represented as a graph in which the **vertices** are states and the **directed edges** between them are actions.



Toy Examples : the 8-puzzle



Start State



Goal State

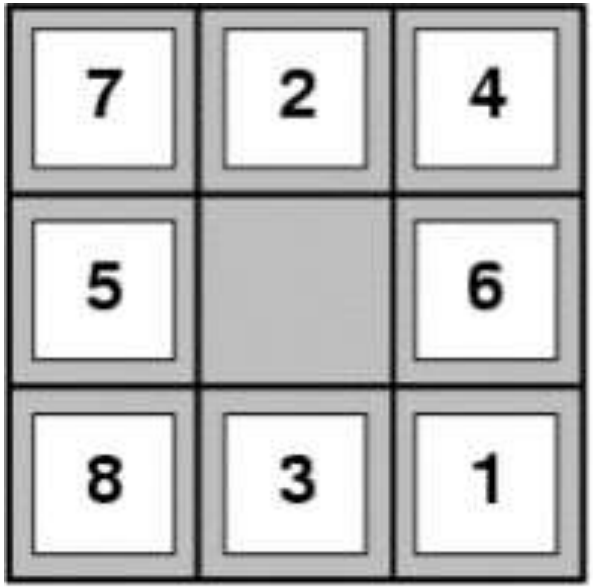
states?

actions?

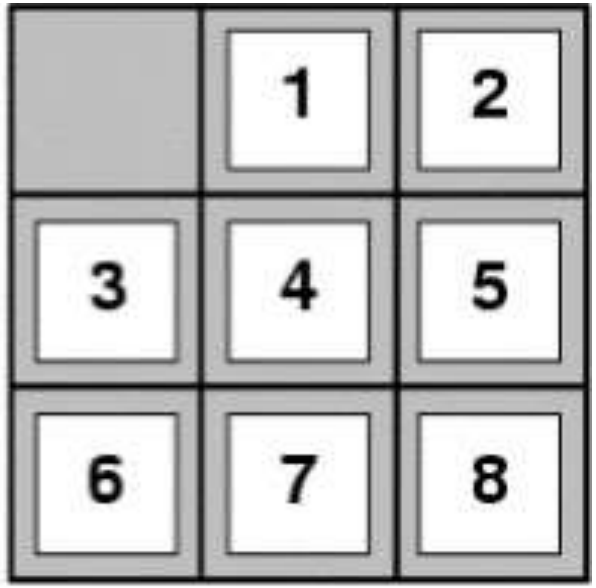
goal test?

path cost?

Example: The 8-puzzle



Start State



Goal State

states? locations of tiles

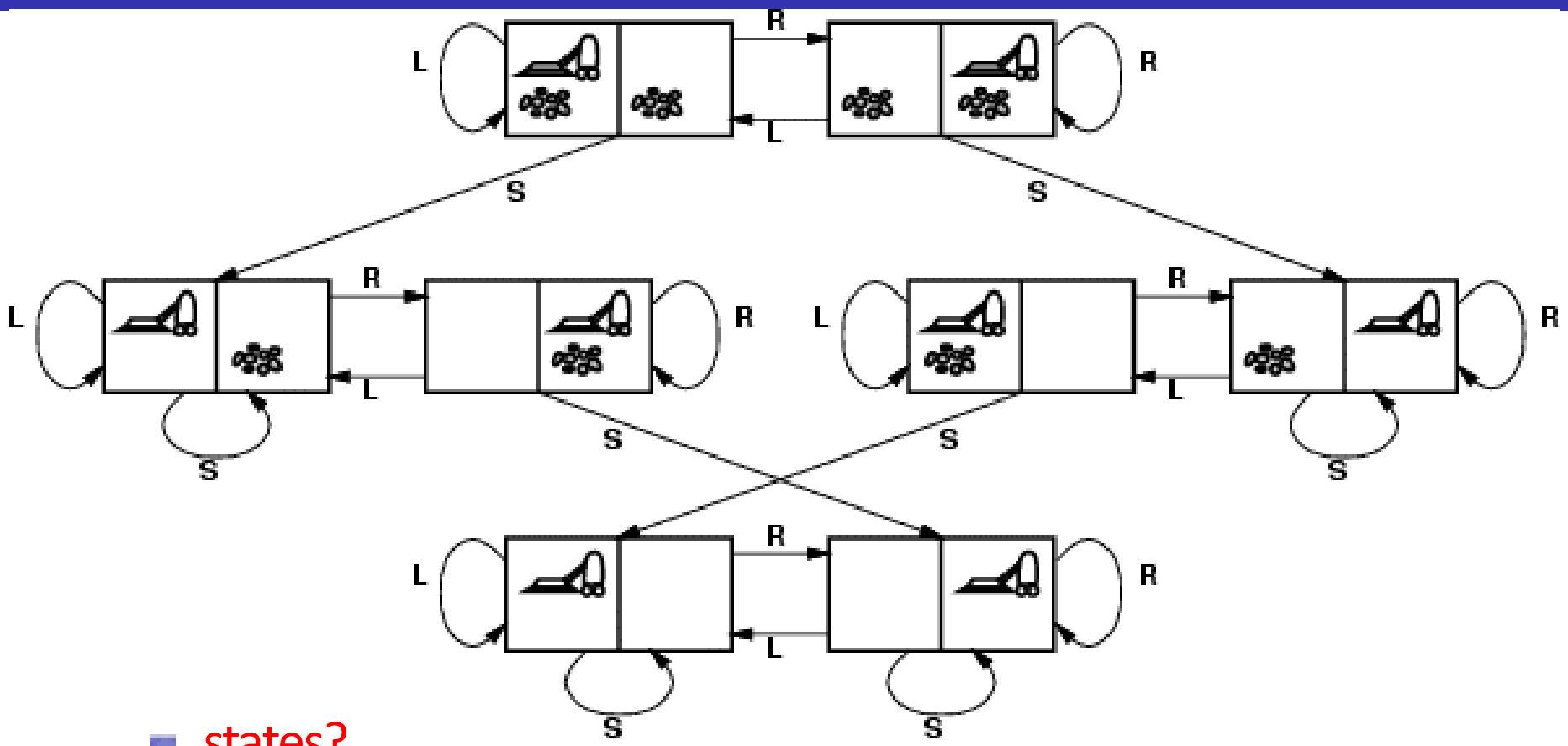
actions? move blank left, right, up, down

goal test? = goal state (given)

path cost? 1 per move

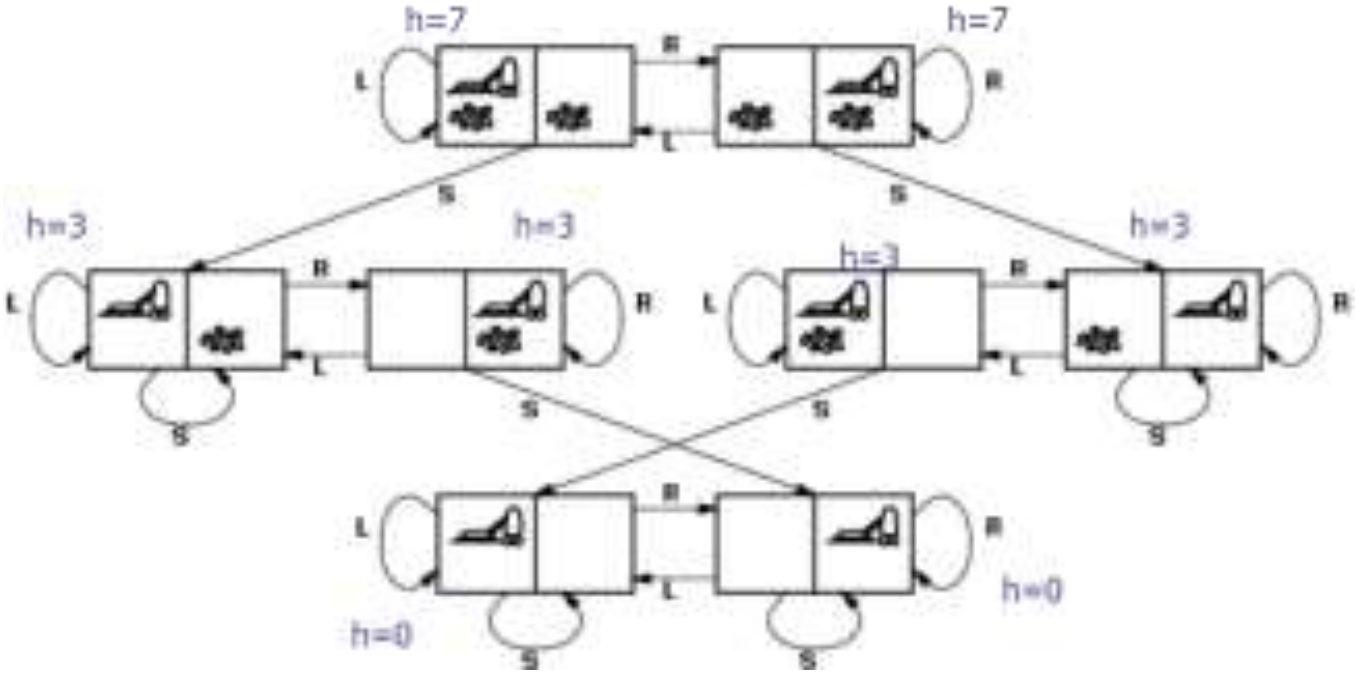
[Note: optimal solution of n-Puzzle family is NP-hard]

Vacuum world state space graph



- states?
- actions (successor function)?
- goal test?
- path cost?

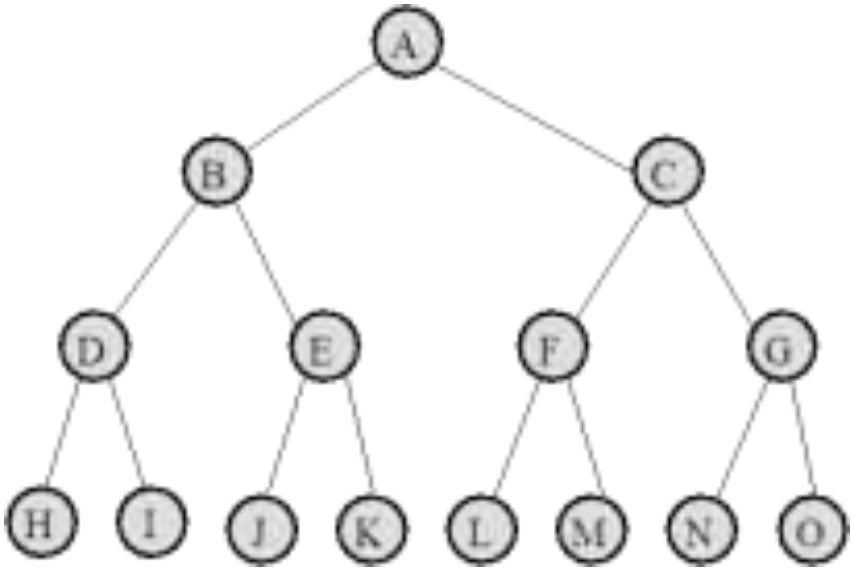
Vacuum world state space graph



- states? integer dirt and robot location
- actions (successor function)?)? Left, Right, Suck
- goal test? no dirt at all locations
- path cost? 1 per action

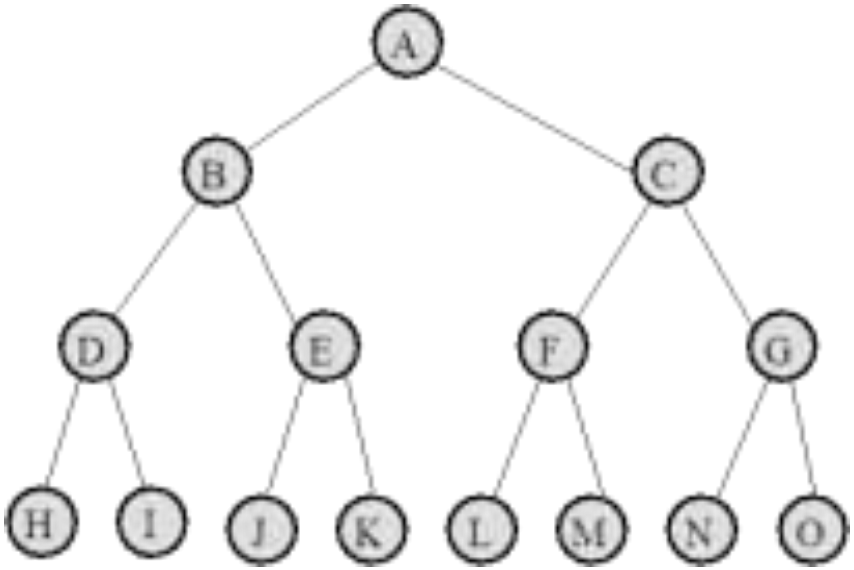
What is Searching in AI?

In computer science, a search algorithm, is an algorithm that takes a problem as input and returns a solution to the problem, usually after evaluating a number of possible solutions.



What is Searching in AI?

In computer science, a search algorithm, is an algorithm that takes a problem as input and returns a solution to the problem, usually after evaluating a number of possible solutions.



AI applications

Many AI problems can be posed as search:

- Touring problem (route-finding problem)
- The travelling salesman problem (TSP)
- Planning
- Robot navigation
- Game-playing
- Speech Recognition
- Resource allocation

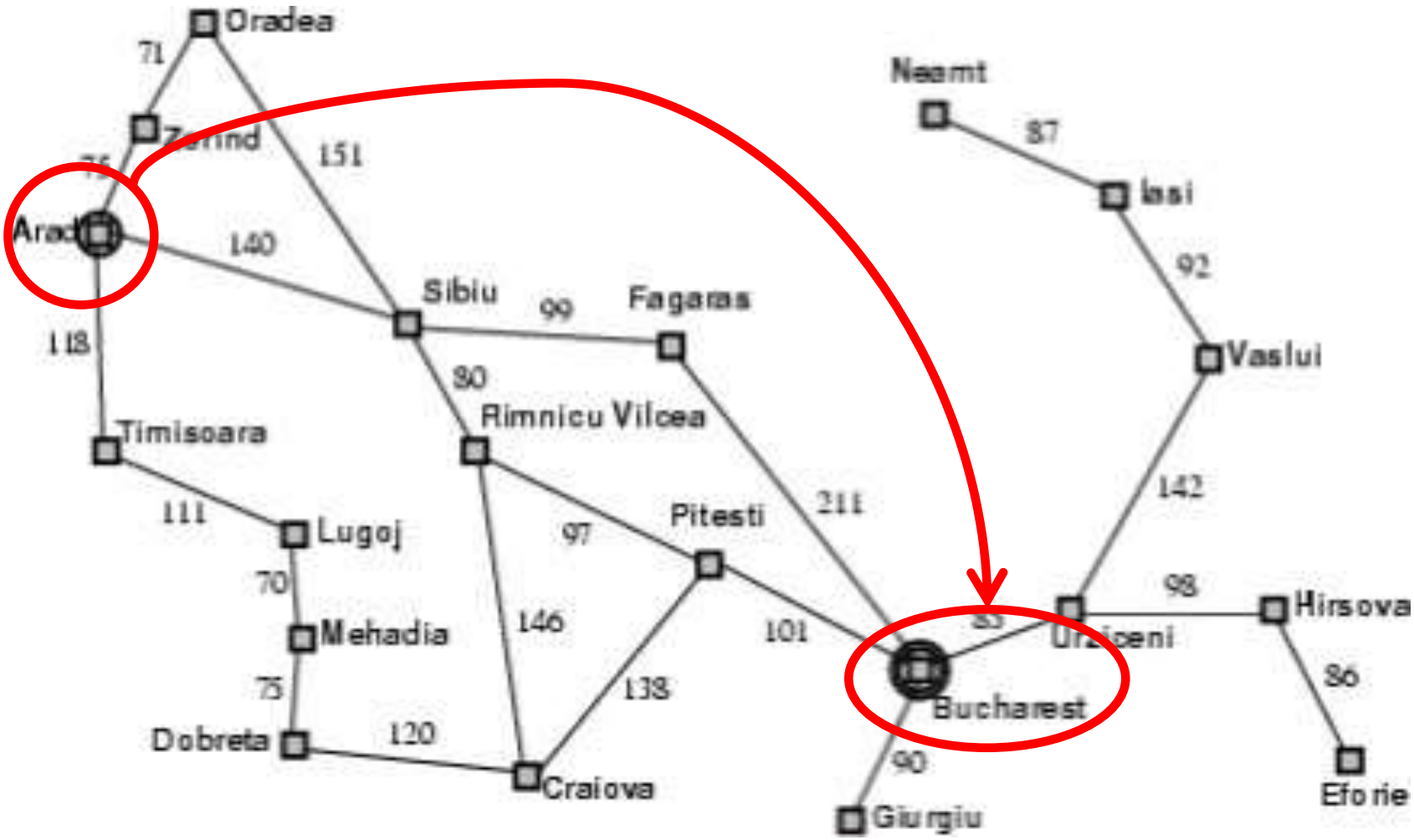
Search tree algorithms

Basic idea:

- exploration of state space by generating successors of already-explored states (expanding states)

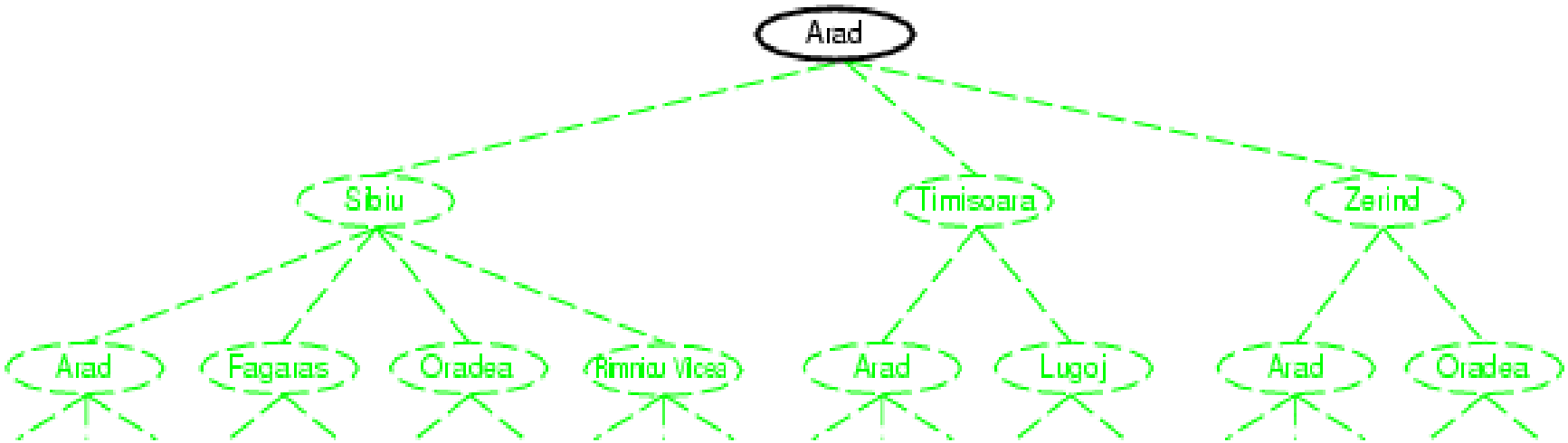
```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
```

Example: Romania



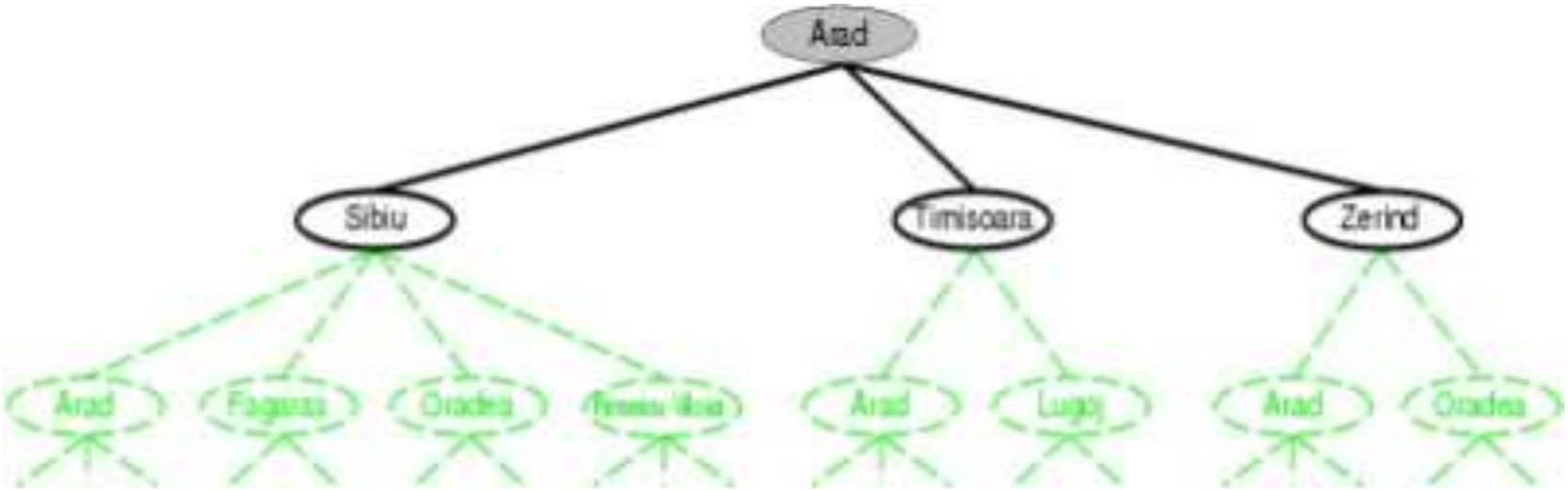
Search tree example

The initial state



Search tree example

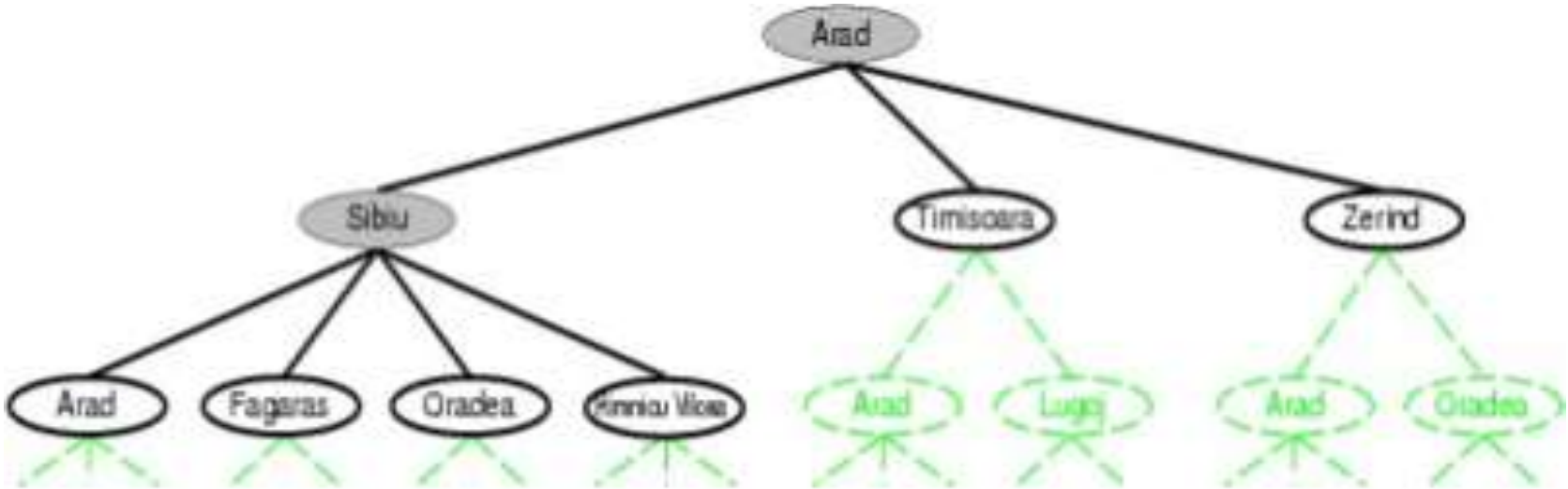
After expanding Arad



Fringe: the set of all nodes at the end of all visited paths is called the fringe, frontier or border. E.g. Sibiu, Timisoara, and Zerind.

Search tree example

After expanding Sibiu



Implementation: general tree-search

```
function TREE-SEARCH( problem, fringe) returns a solution, or failure
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
        fringe ← INSERTALL(EXPAND(node, problem), fringe)

function EXPAND( node, problem) returns a set of nodes
    successors ← the empty set
    for each action, result in SUCCESSOR-FN[problem](STATE[node]) do
        s ← a new NODE
        PARENT-NODE[s] ← node;  ACTION[s] ← action;  STATE[s] ← result
        PATH-COST[s] ← PATH-COST[node] + STEP-COST(node, action, s)
        DEPTH[s] ← DEPTH[node] + 1
        add s to successors
    return successors
```

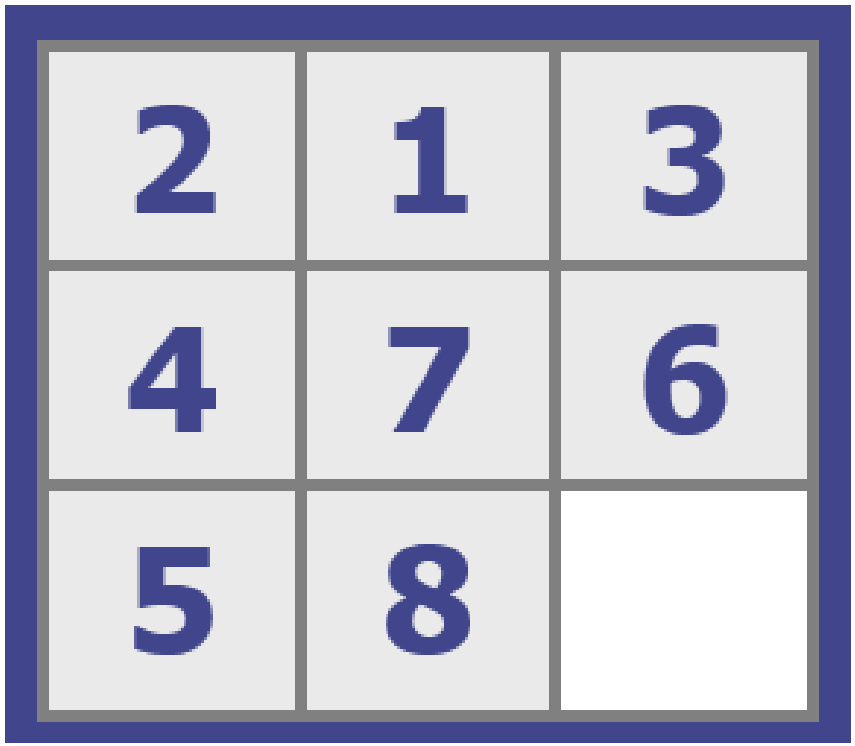
Branching Factors in the search tree

b = branching factor : the number of children at each node. If this value is not uniform, an average branching factor can be calculated.

Branching Factors in the search tree

Example 1:

- The eight puzzle has a branching factor of 2.13, so a search tree at depth 20 has about 3.7 million nodes. (note that there are only 181,400 different states).



Branching Factors in the search tree

Example 2:

- Rubiks cube has a branching factor of 13.34. There are 901,083,404,981,813,616 different states.

  The average depth of a solution is about 18.
- The best time for solving the cube in an official championship was 17.04 sec, achieved by Robert Pergl in the 1983 Czechoslovakian Championship. In 1997 the best AI computer programs took weeks.





Open AI



MIT robot: 0.38 sec

Branching Factors in the search tree

Example 3:

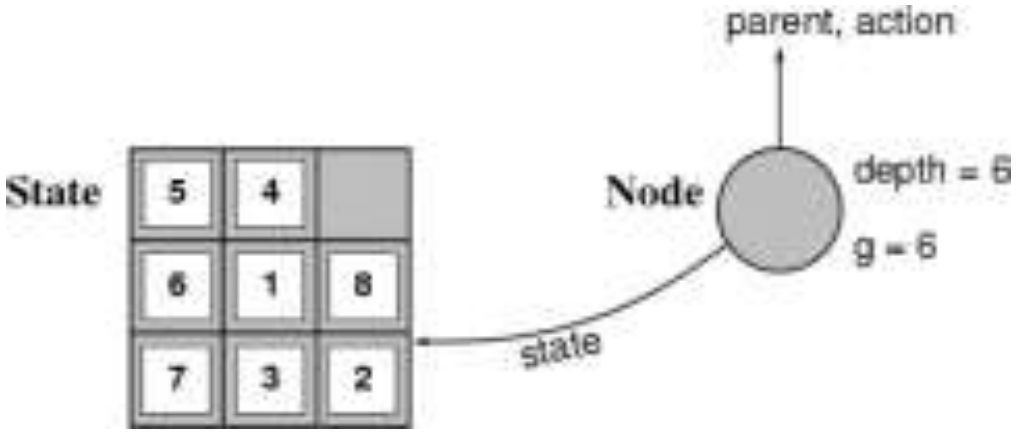
- Chess has a branching factor of about 35, this means that, on average, a player has about 35 legal moves at his disposal at each turn.





Implementation: states vs. nodes

- A state is a (representation of) a physical configuration. In a state space graph, each state occurs only once!
- A node is a data structure constituting part of a search tree includes state, parent node, action, path cost g(x), depth



- The *Expand* function creates new nodes, filling in the various fields and using the *SuccessorFn* of the problem to create the corresponding states.

Search strategies

- A search strategy is defined by picking the order of node expansion.

  Strategies are evaluated along the following dimensions:

  - completeness: does it always find a solution if one exists?
  - time complexity: number of nodes generated.
  - space complexity: maximum number of nodes in memory.
  - optimality: does it always find a least-cost solution?

- Time and space complexity are measured in terms of

  - *b:* maximum branching factor of the search tree
  - *d:* depth of the least-cost solution (the depth of the shallowest goal)
  - *m:* maximum depth of the state space

Types of Search Techniques

- Uninformed Search Techniques (Blind Search)
  - Breadth-First Search (BFS)
  - Uniform Cost Search (UCS)
  - Depth-First Search (DFS)
  - Depth Limited Search (DLS)
  - Iterative Deepening Search
  - Bi-directional Search

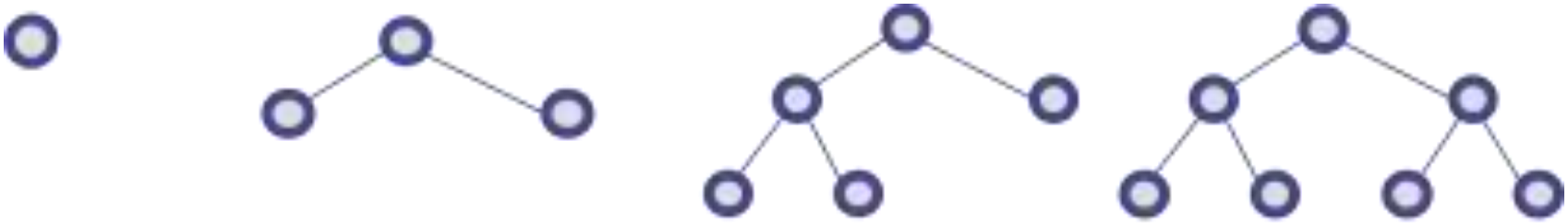
✓ use only the information available in the problem definition.
✓ generate successors and distinguish a goal state from non-goal state.

- Informed Search Techniques (Heuristic Search)
  - Hill Climbing (HC)
  - Best First Search (Best-S)
  - Beam Search
  - The A* Algorithm (A-Star)

✓ strategies that know whether one non-goal state is "more promising" than another or not.

1. Breadth First Search (BFS)

Enqueue nodes in FIFO (first–in, first–out) order.

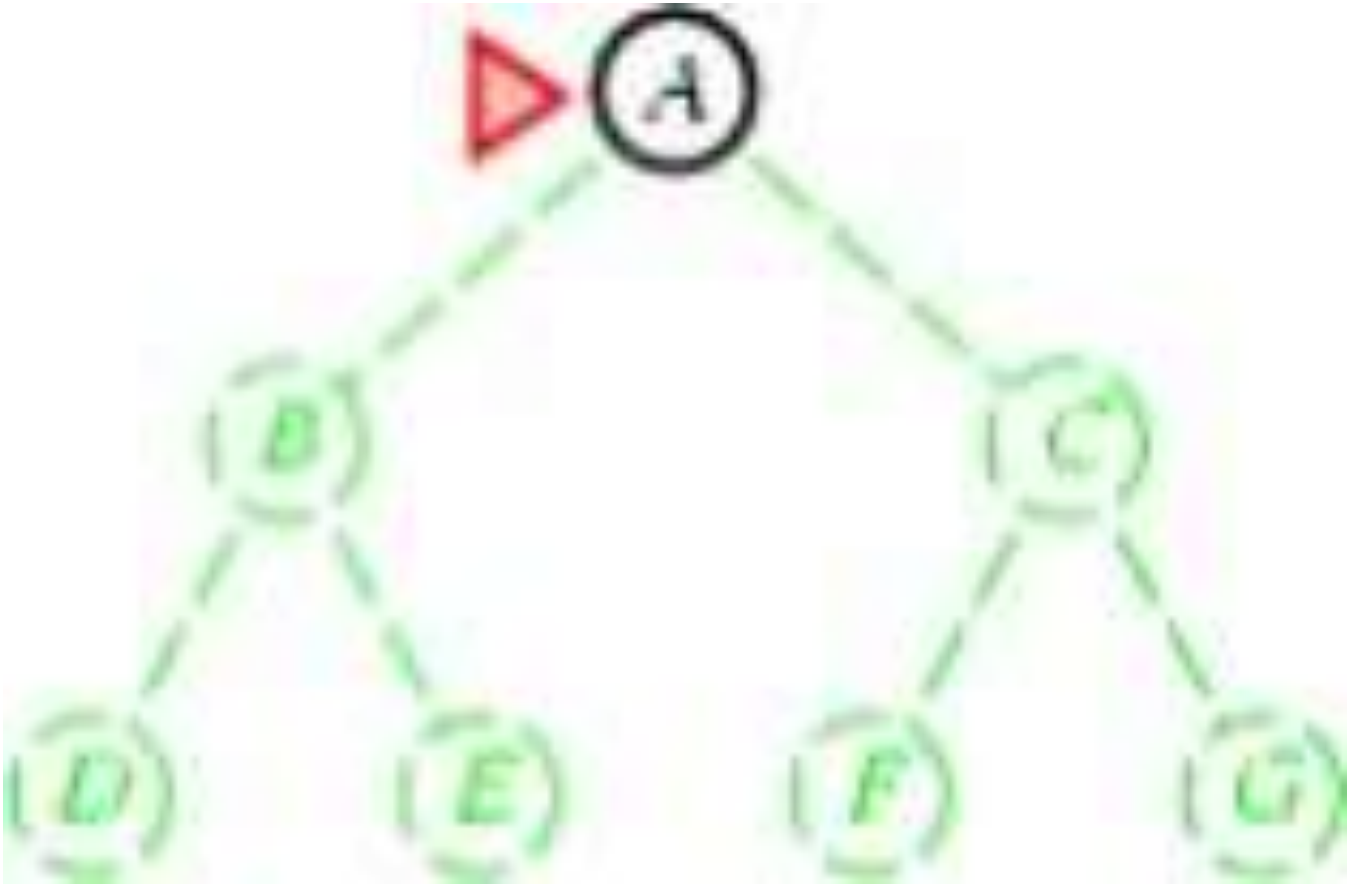


Intuition: Expand all nodes at depth i before expanding nodes at depth $i + 1$.

Breadth-first search

Implementation:

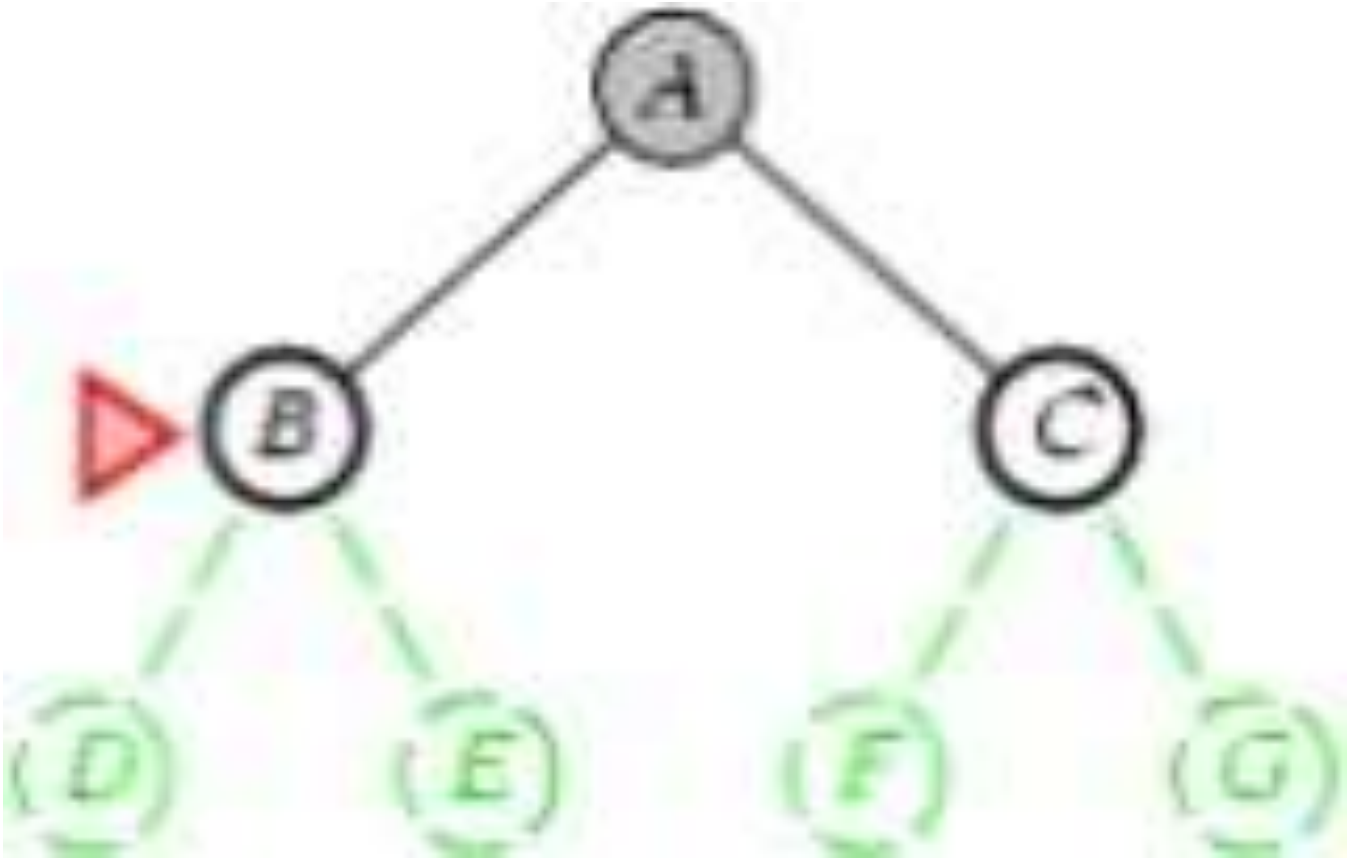
- fringe is a FIFO queue, i.e., new successors go at end



Breadth-first search

Implementation:

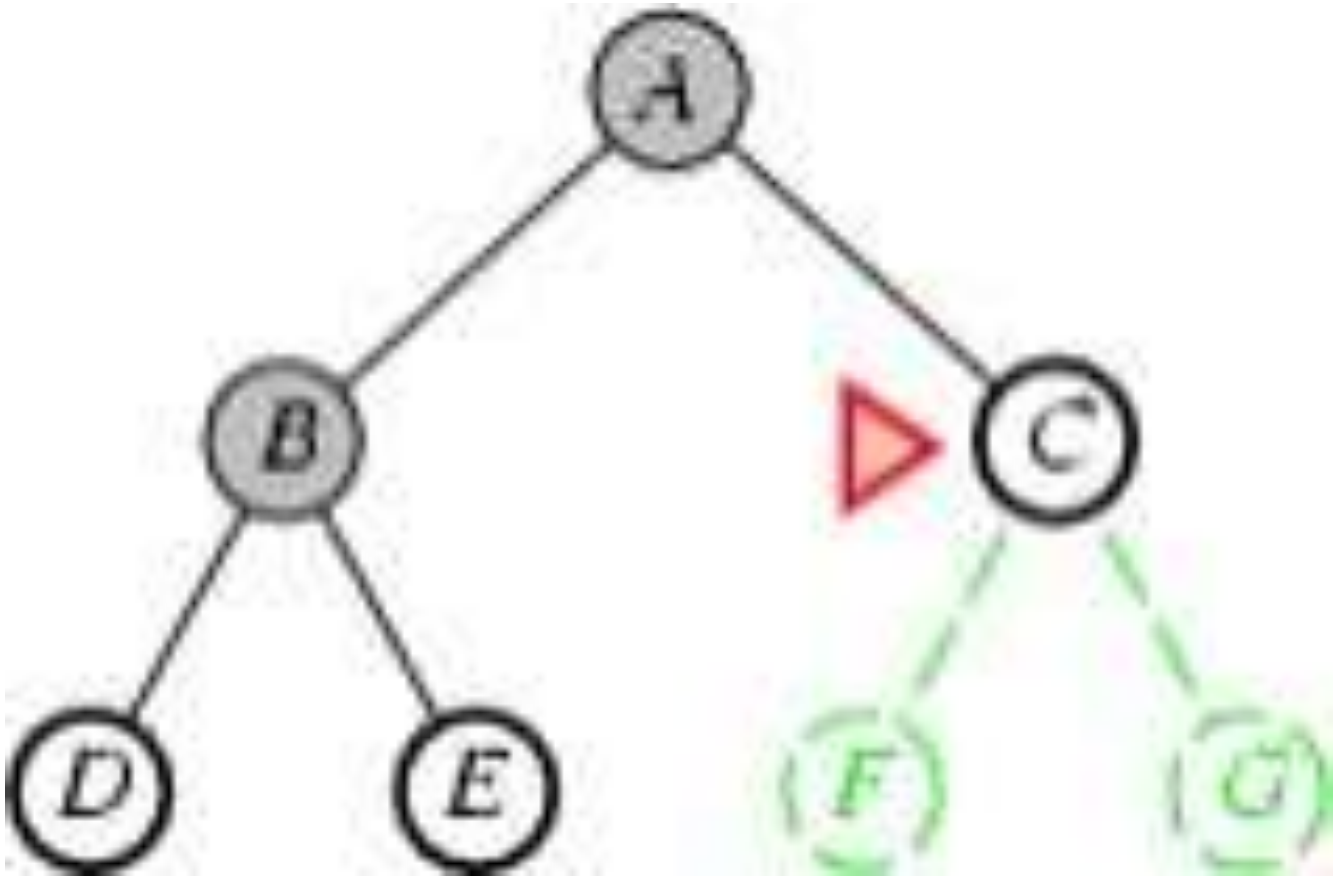
- fringe is a FIFO queue, i.e., new successors go at end



Breadth-first search

Implementation:

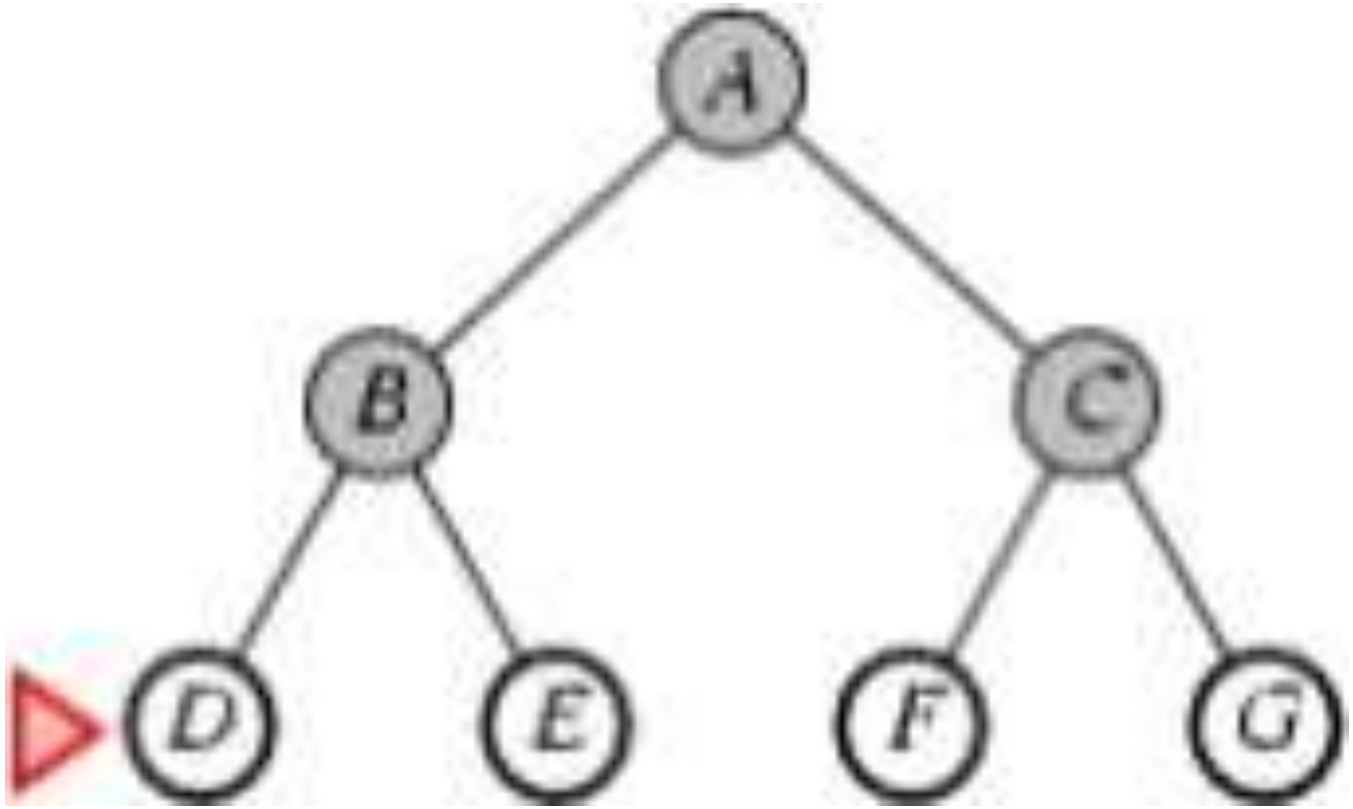
- fringe is a FIFO queue, i.e., new successors go at end



Breadth-first search

Implementation:

- fringe is a FIFO queue, i.e., new successors go at end



1. Breadth First Search (BFS)

- **Time Complexity**: b is branching factor and d is the depth of the solution.

$$T(b) = 1+b^2+b^3+.....+b^d = O(b^d)$$

  - If the algorithm were to apply the goal test to nodes when selected for expansion, rather than when generated, the whole layer of nodes at depth d would be expanded before the goal was detected and the time complexity would be $O(b^{d+1})$.

$$1+b+b^2+b^3+\ldots+b^d+b(b^d-1) = O(b^{d+1})$$

- **Space Complexity:** every node in the fringe is kept in the queue (keeps every node in memory) $O(b^{d+1})$
  - **Space**: is the bigger problem (more than time).

Properties of breadth-first search

- **Complete?** Yes (if b is finite)
- **Optimal** Yes (if costs are equals for all steps).
- **Time?** $1+b+b^2+b^3+\ldots+b^d+b(b^d-1) = O(b^{d+1})$
- **Space?**: $O(b^{d+1})$(keeps every node in memory)

Time and memory requirement in Breadth-first

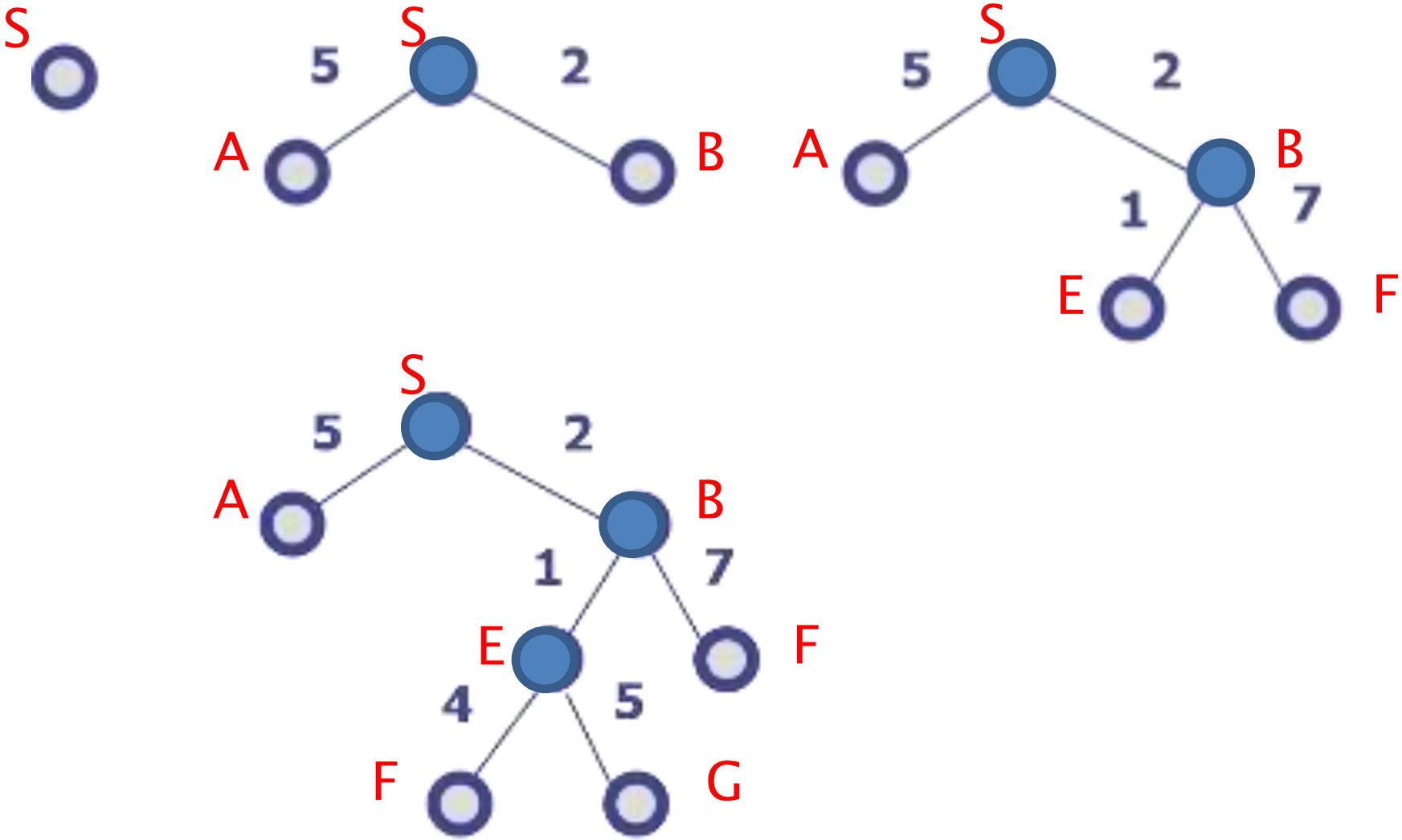
Depth	Nodes	Time	Memory
0	1	1 millisec	100 bytes
2	111	.1 sec	11 Kb
4	11.111	11 sec	1 Mb
6	10^6	18 minutes	111 Mb
8	10^8	31 hours	11 Gb
10	10^{10}	128 days	1 Tb
12	10^{12}	35 years	111 Tb
14	10^{14}	3500 years	11.111 Tb

Assume branching factor b=10; 1000 nodes explored/sec and 100 bytes/node

2.Uniform Cost Search (UCS)

- Enqueue nodes in order of cost.
- **Intuition**: Expand the cheapest node, where the cost is **the path cost *g(n)***. This is done by storing the frontier as a priority queue ordered by *g* instead of simple queue as in BFS.
- Equivalent to BFS if step costs are all equal.

2.Uniform Cost Search (UCS)



2.Uniform Cost Search (UCS)

- **Complete?** Yes, if b is finite and all step costs are positives.
  - Uniform-cost search does not care about the number of steps a path has, but only about their **total cost**.
  - It will get stuck in an infinite loop if there is a path with an infinite sequence of zero-cost actions—for example, a sequence of NoOp actions.

- **Optimal?** Yes (all step costs are positives.)

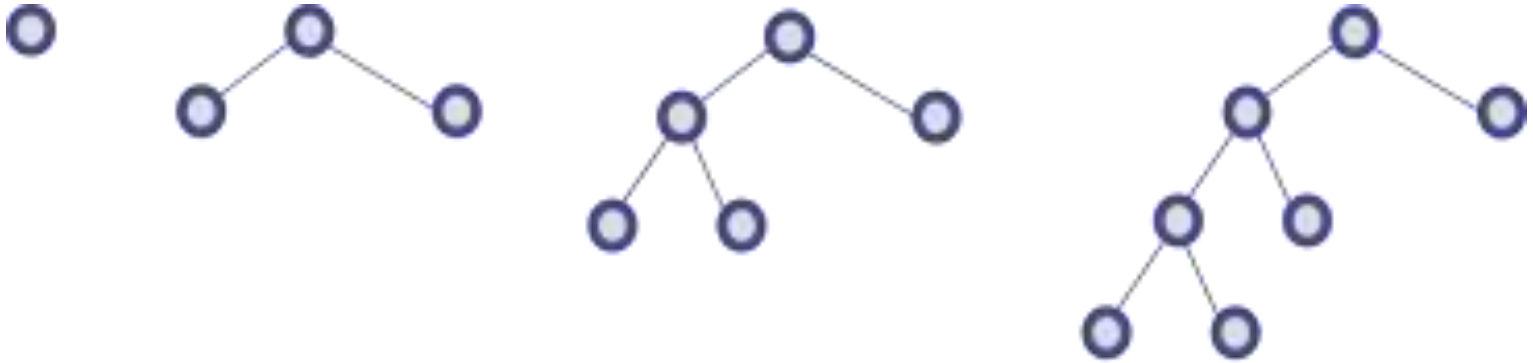
2.Uniform Cost Search (UCS)

Time and space Complexity:

- Uniform-cost search is guided by **path costs** rather than depths, so its complexity is not easily characterized in terms of **b** and **d**.
- let C^* be the **cost of the optimal solution** and assume that every action costs at least ϵ (path cost always increases as we go along the path).
- Then the algorithm's **worst-case time and space complexity** is $O(b^{1+\lfloor C^*/\epsilon \rfloor})$, which can be much larger than b^d
- When **all step costs are equal**, time complexity is $O(b^{d+1})$.

3.Depth First Search (DFS)

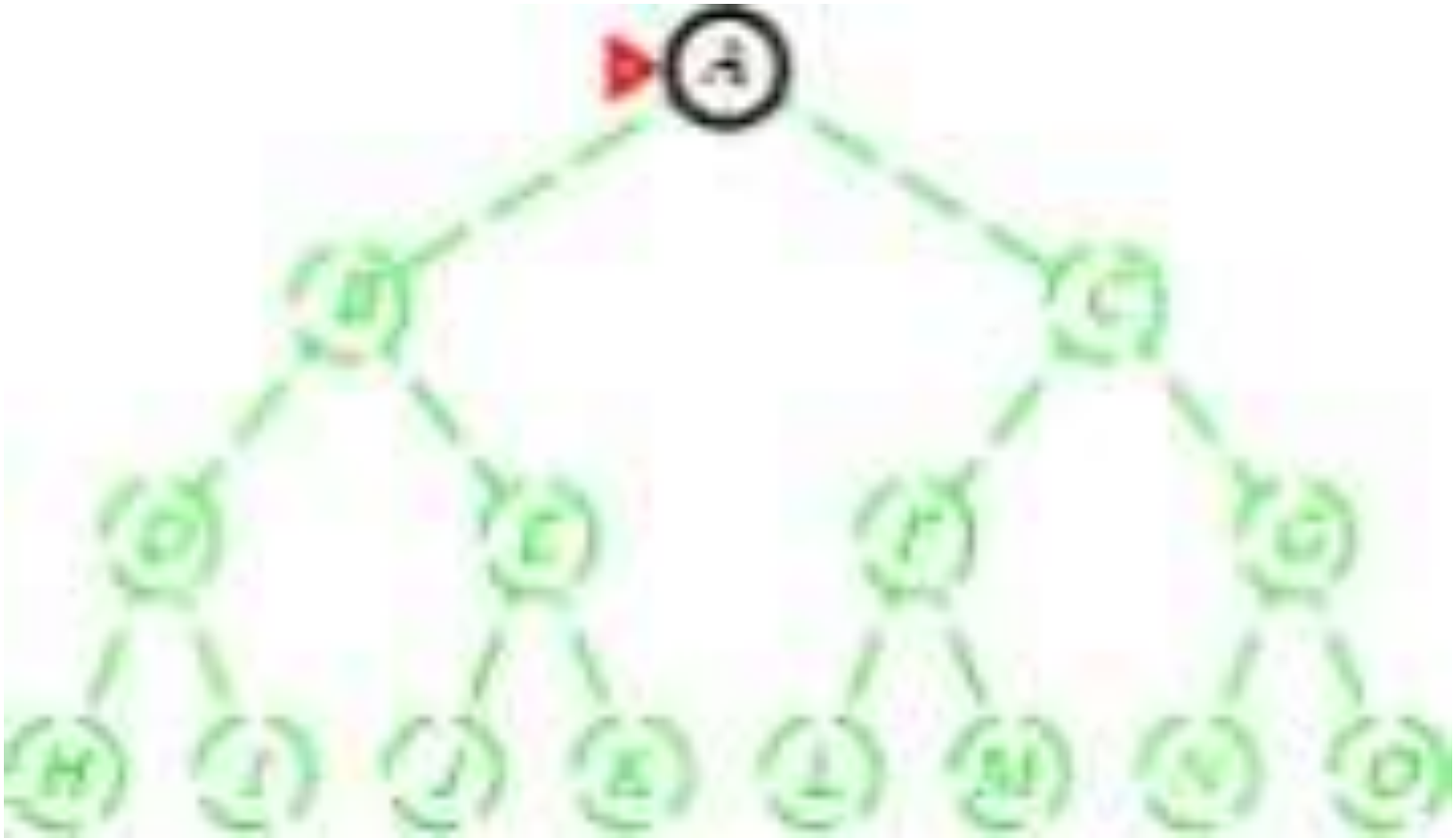
- Enqueue nodes in LIFO (last-in, first-out) order.



- **Intuition:** Expand node at the deepest level (breaking ties left to right).

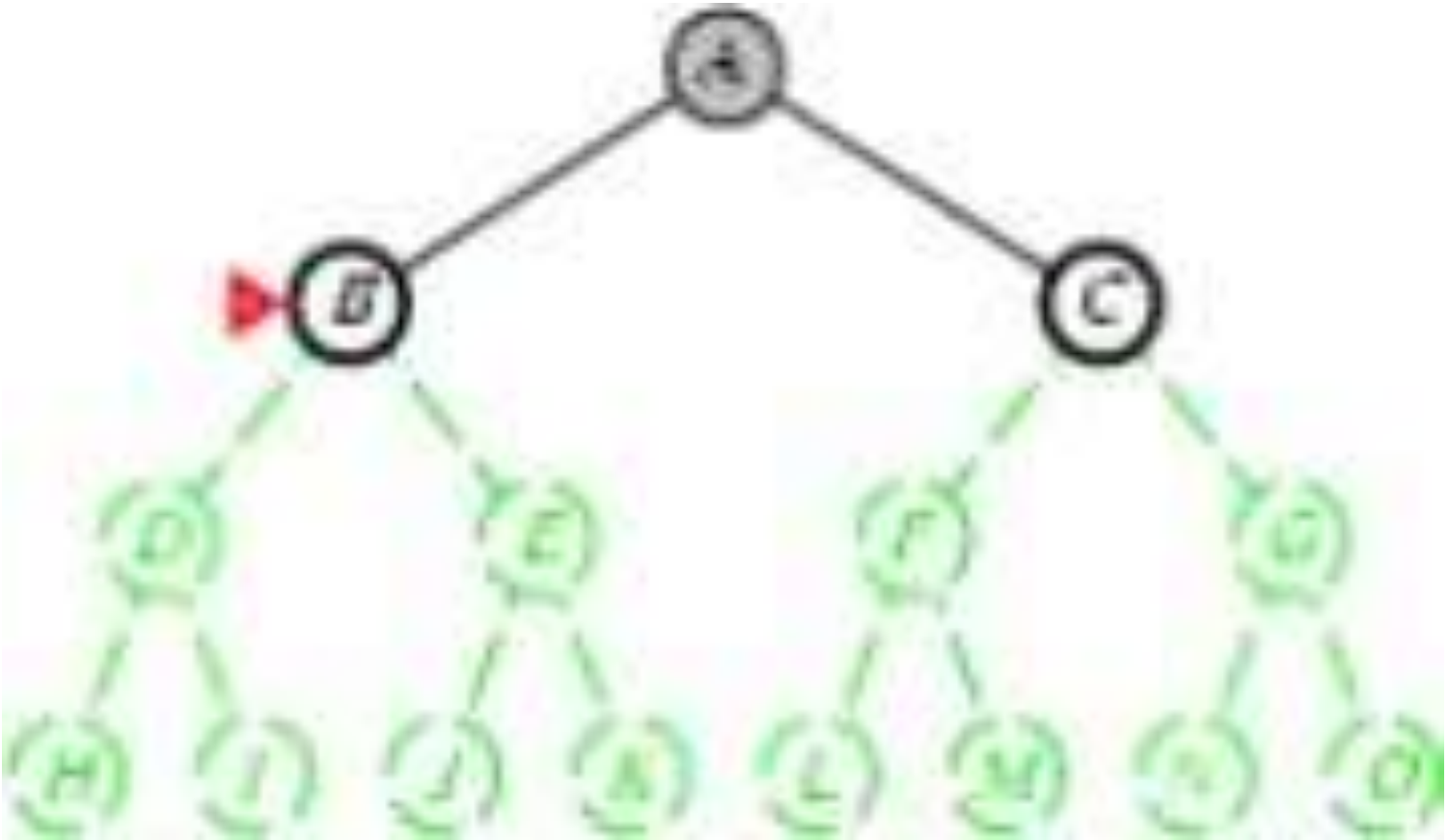
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



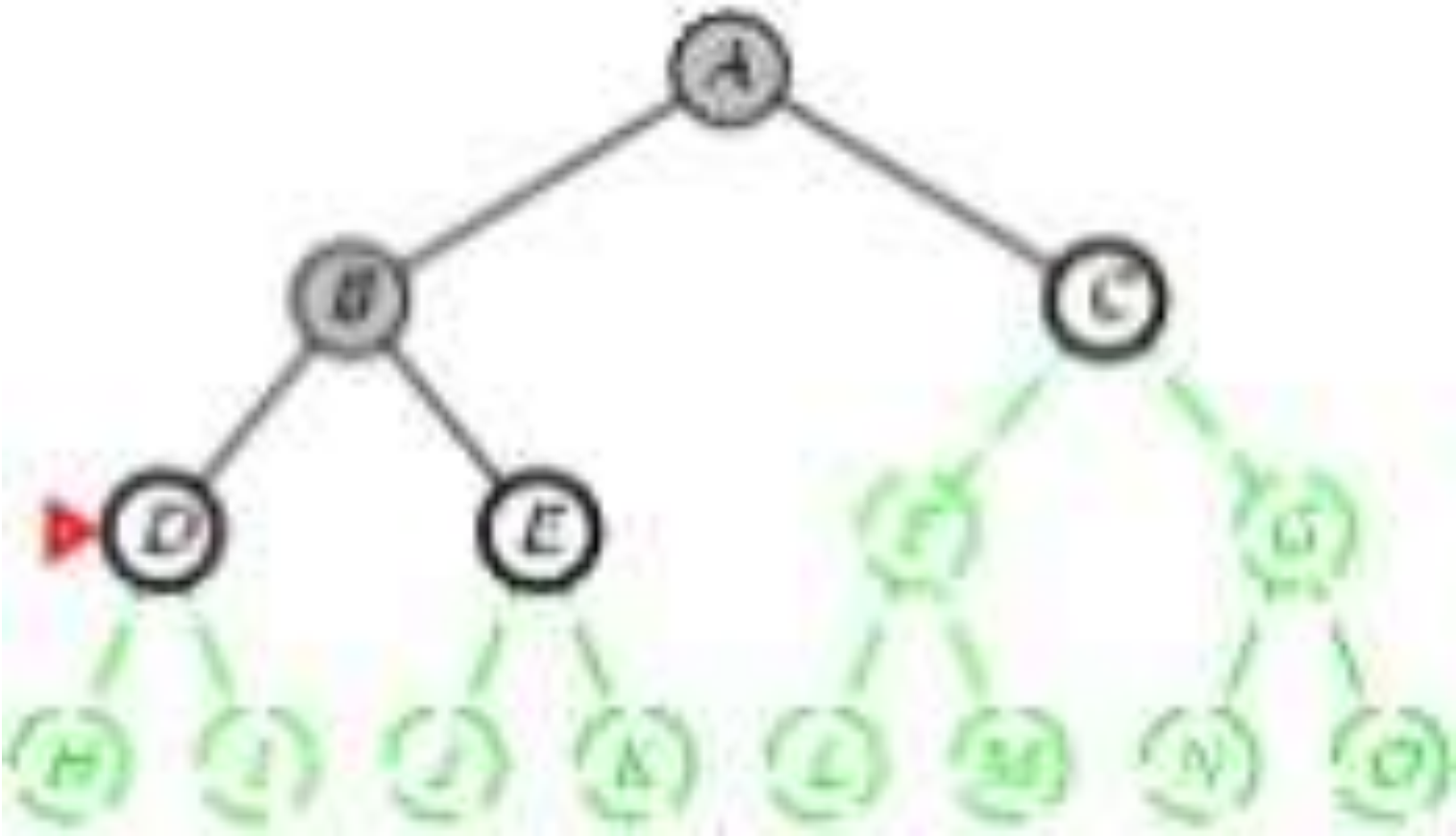
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



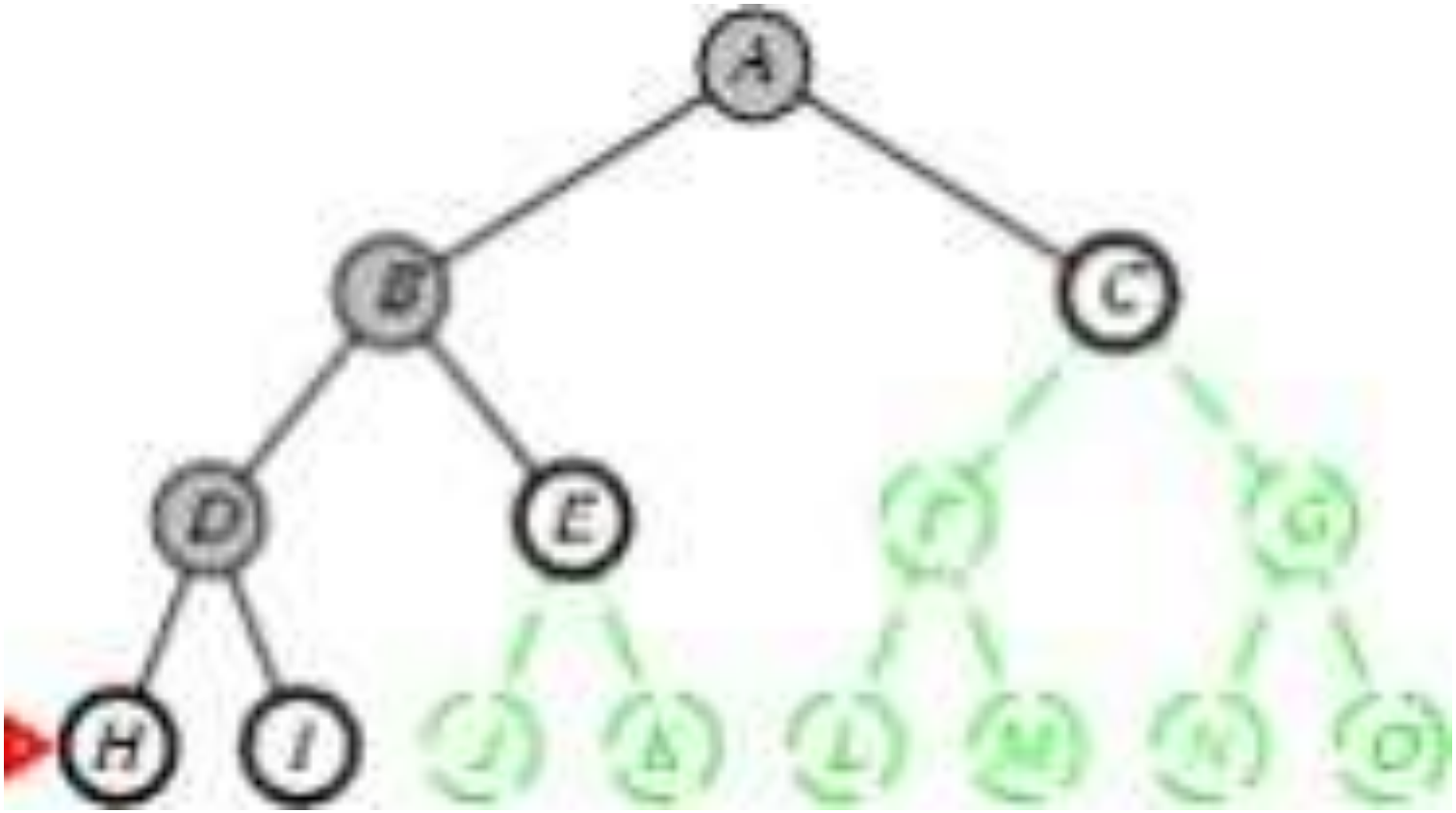
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



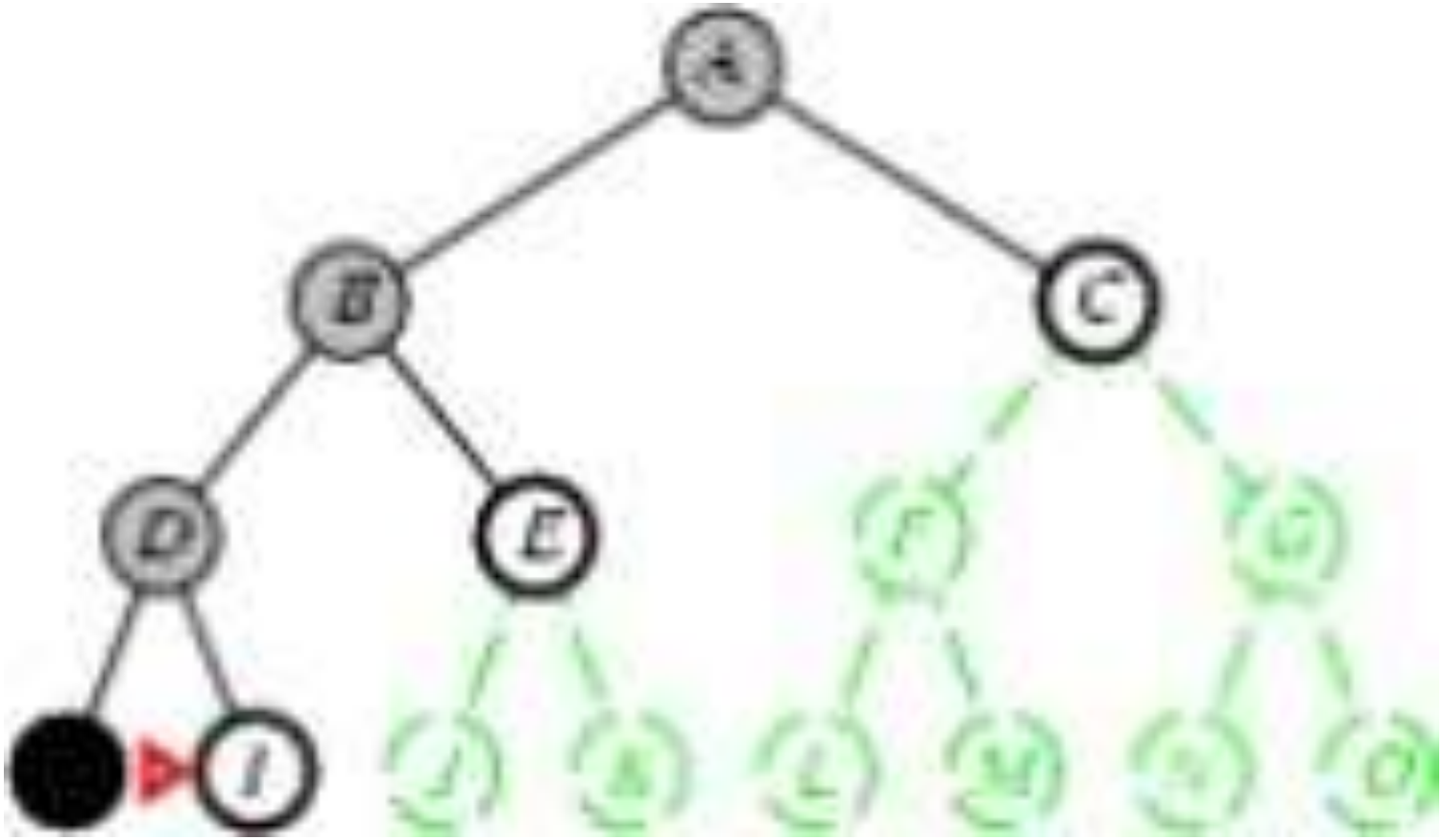
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



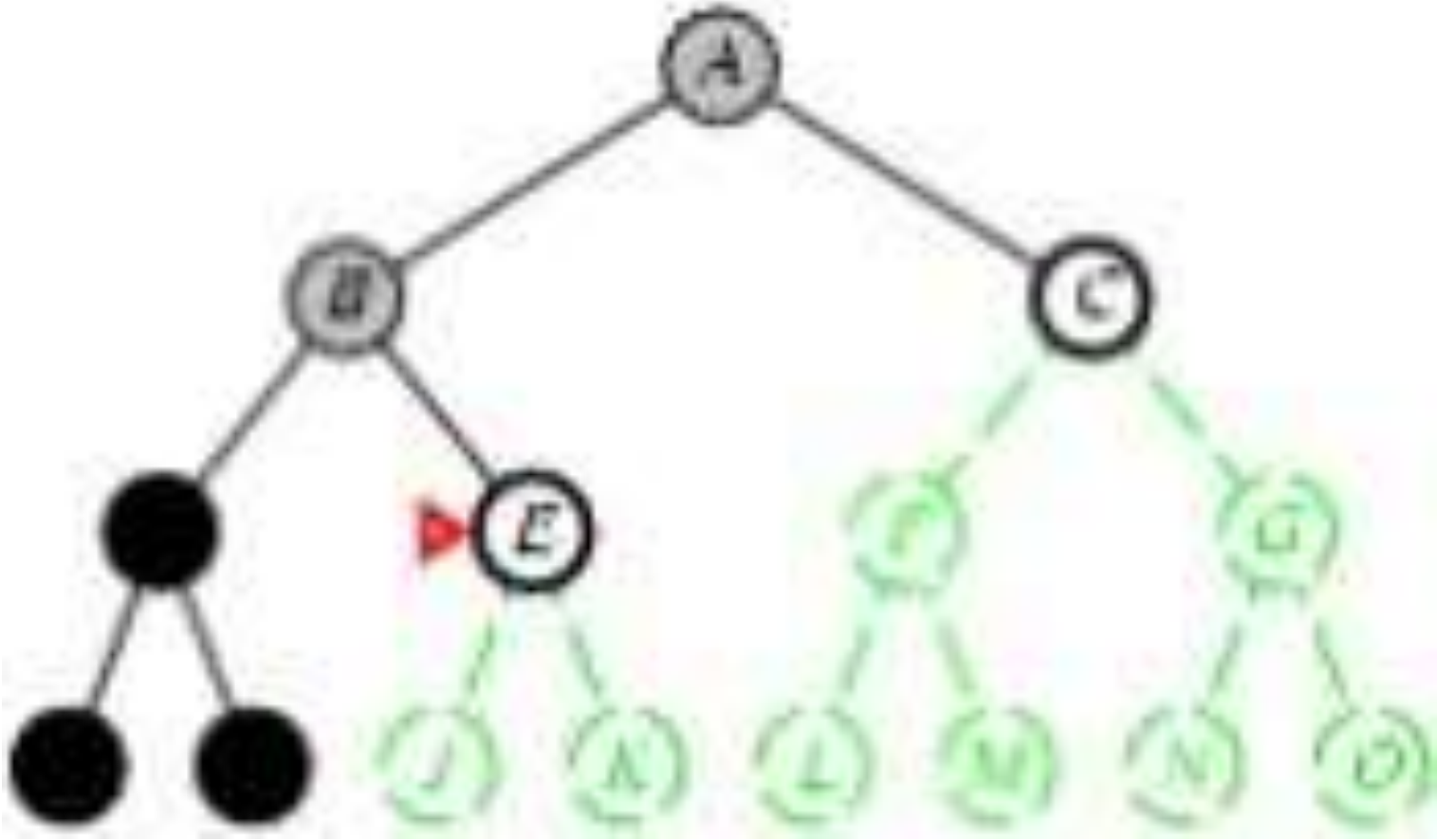
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



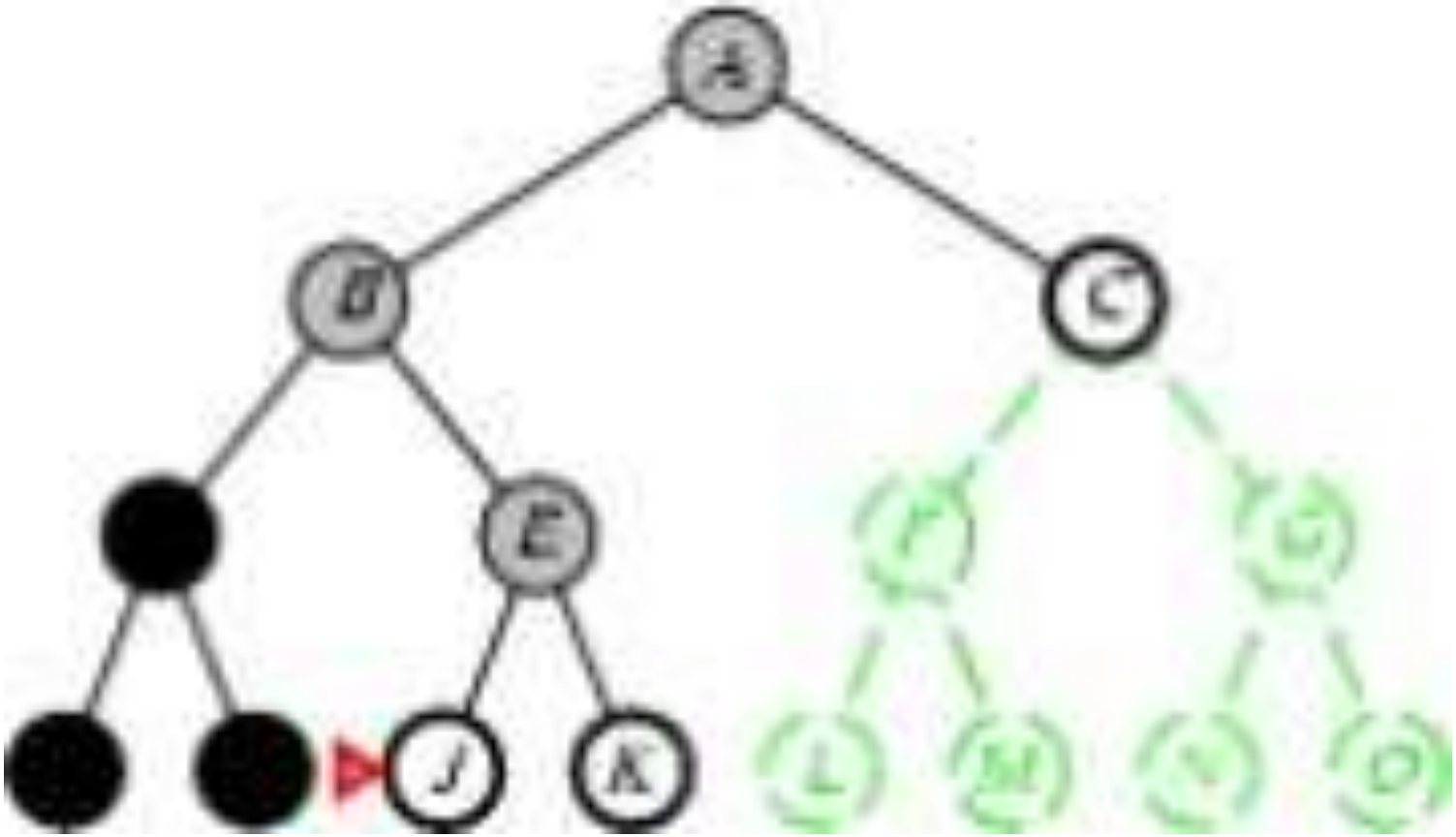
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



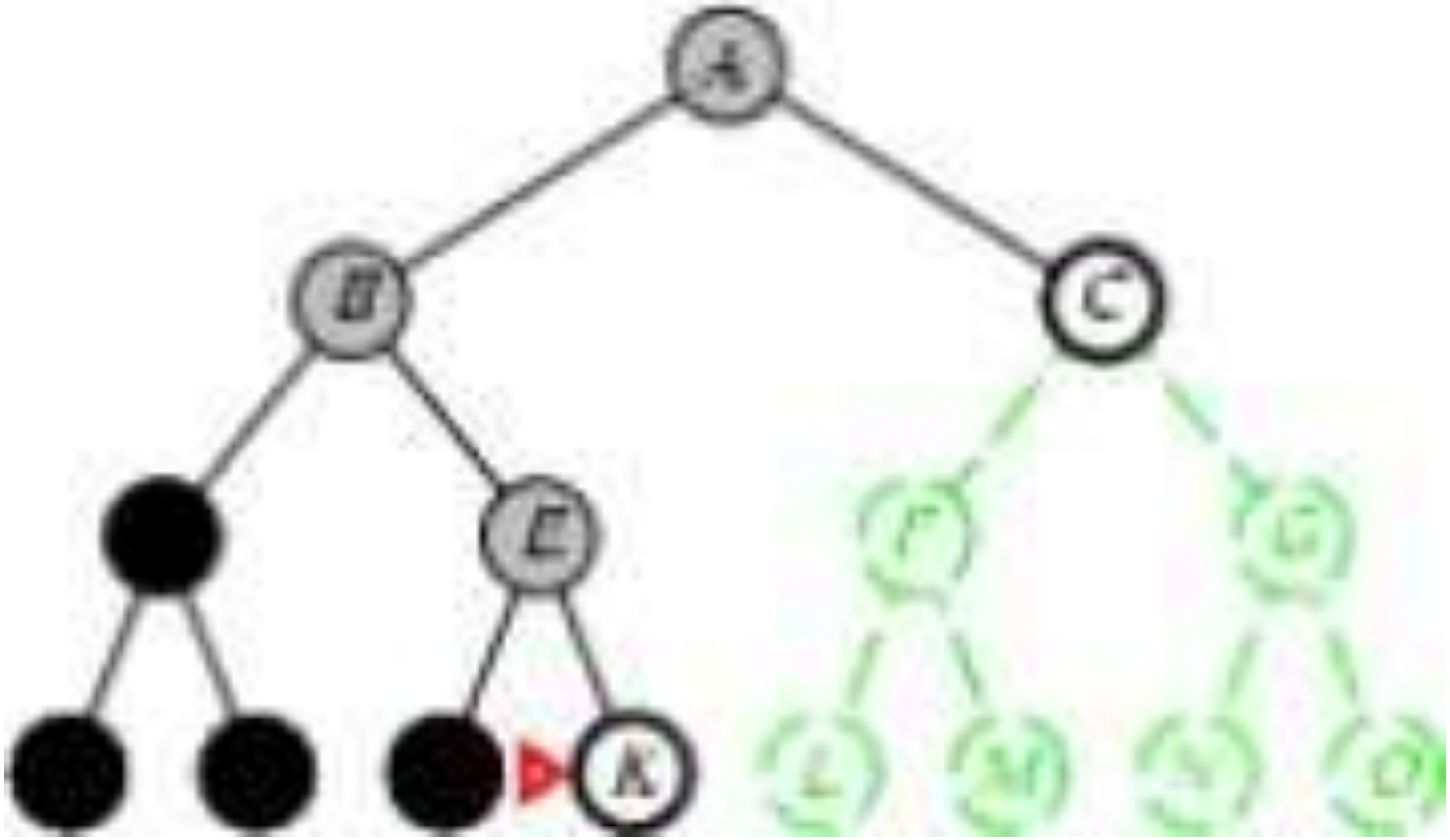
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



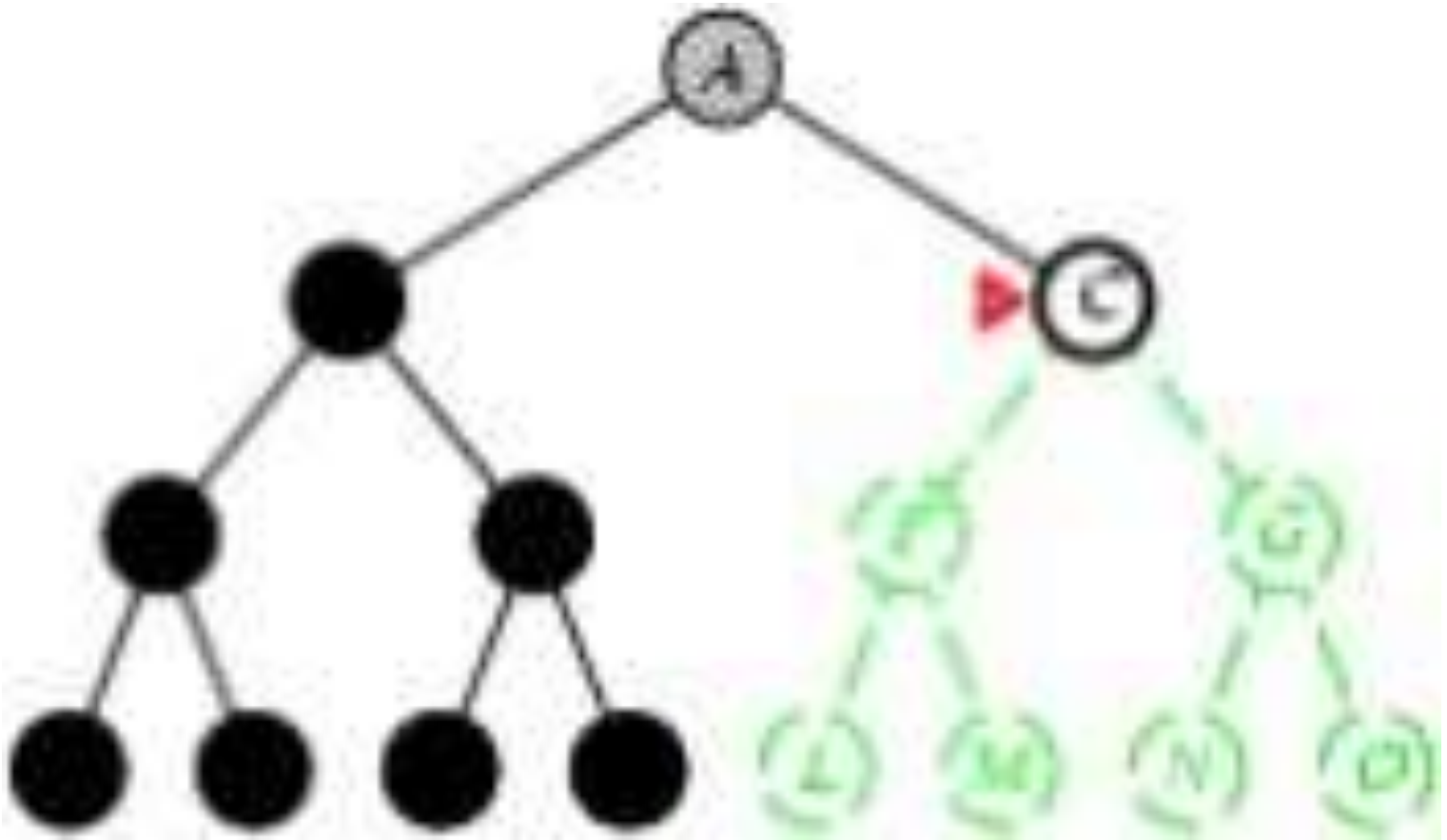
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



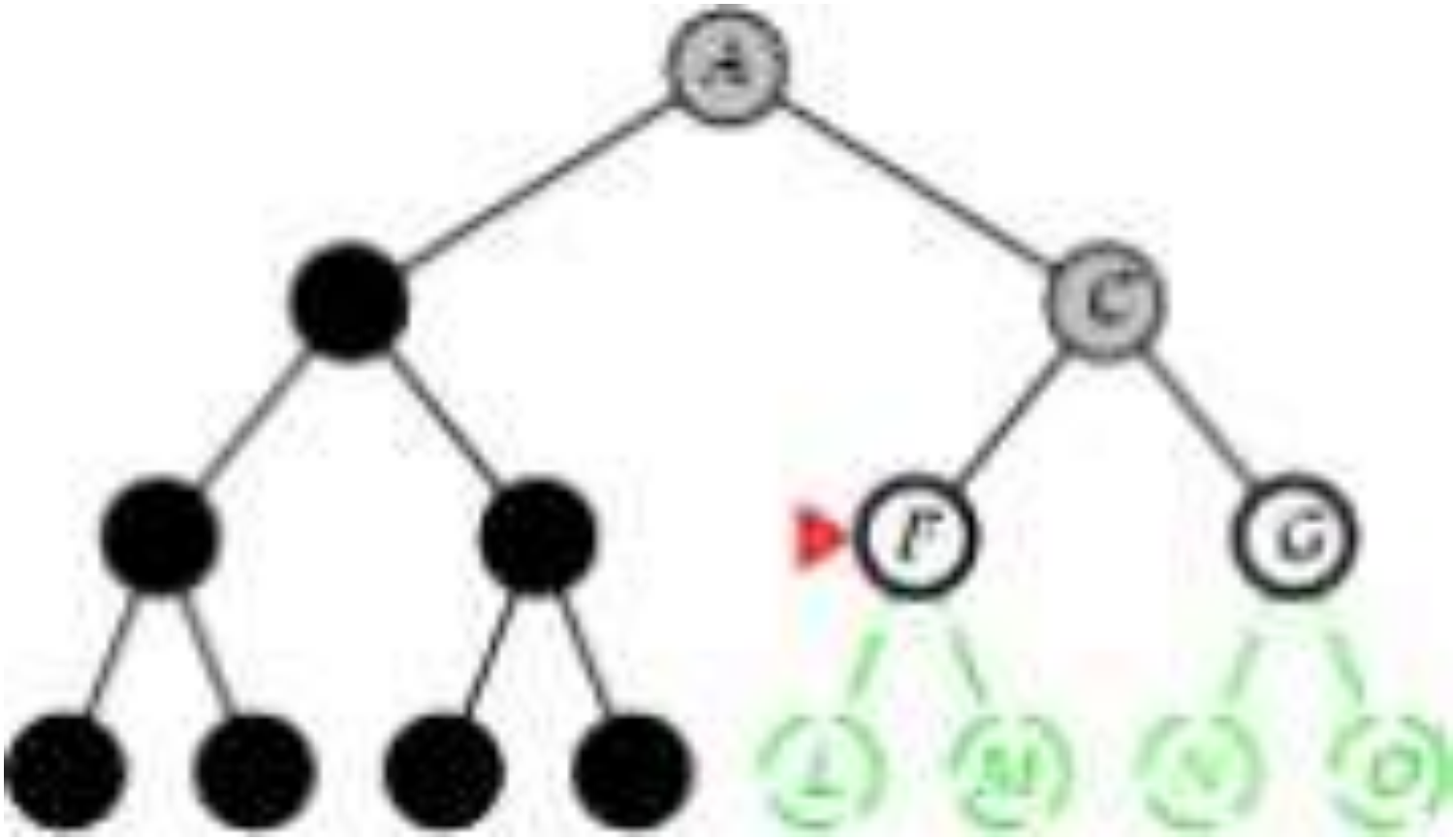
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



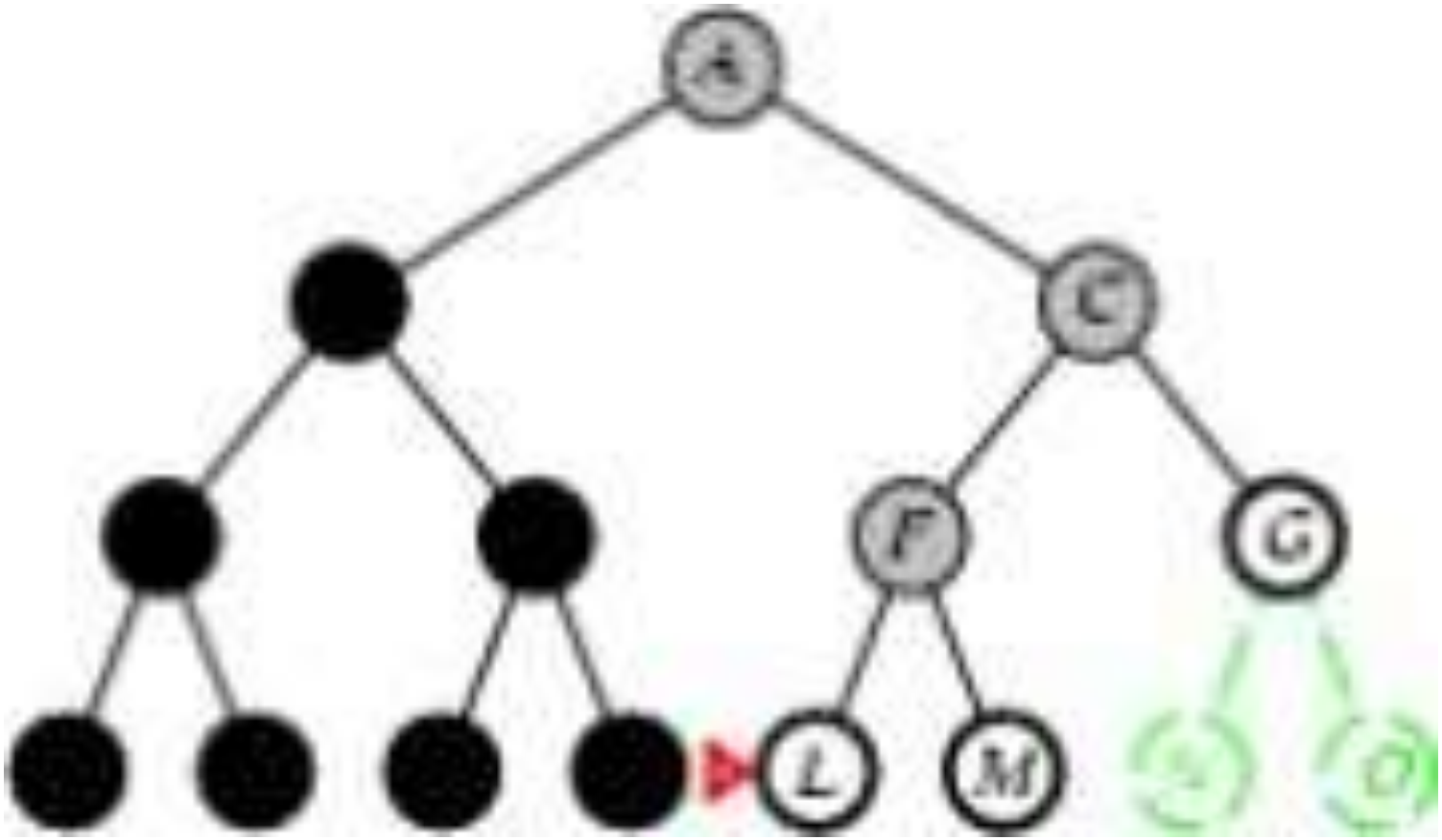
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



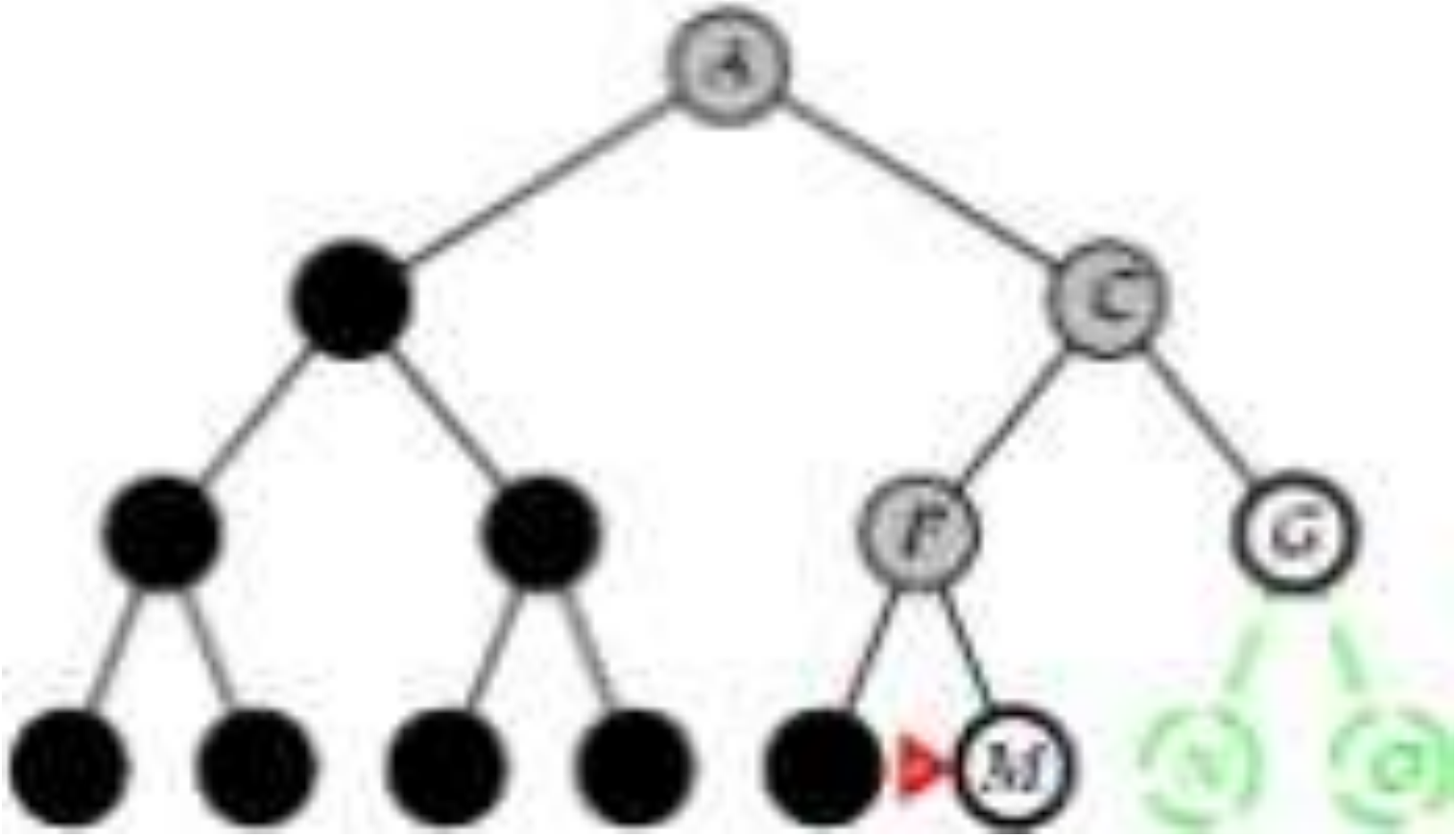
Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



Depth-first search

- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



Properties of depth-first

- Complete? No: fails in infinite-depth spaces, spaces with loops
  - Modify to avoid repeated states along path
    - Complete in finite spaces
- Time? $O(b^m)$: terrible if m is much larger than d
  - but if solutions are dense, may be much faster than breadth-first
- Space? O(bm), i.e., linear space!
- Optimal? No.

Time and memory requirement in Depth-first

Depth	Nodes	Time (best case)	Memory
0	1	1 millisec	100 bytes
2	20	0.02 sec	2 Kb
4	40	0.04 sec	4 Kb
6	10 * 6	0.06 sec	6 Kb
8	10 * 8	0.08 sec	8 Kb
10	10 *10	0.1 sec	10 Kb
12	10 * 12	0.12 sec	12 Kb
14	10 * 14	0.14 sec	14 Kb

Assume branching factor b=10; 1000 nodes explored/sec and 100 bytes/node

Avoiding reapeted states : Graph

- If a strategy remembers every state that it has visited, then it can viewed as exploring the **state-space graph** directly.
- **Closed list** data structure which stores every expanded node.
- If the current node matches a node on **closed list**, it is discarded instead of being expanded.

Graph search

```
function GRAPH-SEARCH( problem, fringe) returns a solution, or failure
    closed ← an empty set
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
        if STATE[node] is not in closed then
            add STATE[node] to closed
            fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

Summary of algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First
Complete?	Yes[a]	Yes[a,b]	No
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$
Optimal?	Yes[c]	Yes	No

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: [a] complete if b is finite; [b] complete if step costs $\geq \epsilon$ for positive ϵ; [c] optimal if step costs are all identical;